University of Reading
Department of Computer Science

Computer Science Undergraduate Report

Felix Possoz-Beech
Supervisor: Muhammad Shahzad

A report submitted in partial fulfilment of the requirements of
the University of Reading for the degree of

Bachelor of Science in Computer Science
May 14 2026

Declaration

I, Felix Possoz-Beech, of the Department of Computer Science, University of Reading, confirm that this is my own work and figures, tables, equations, code snippets, artworks, and illustrations in this report are original and have not been taken from any other person's work, except where the works of others have been explicitly acknowledged, quoted, and referenced. I understand that if failing to do so will be considered a case of plagiarism. Plagiarism is a form of academic misconduct and will be penalised accordingly.
I give consent to a copy of my report being shared with future students as an exemplar.
I give consent for my work to be made available more widely to members of UoR and public with interest in teaching, learning and research.

Felix Possoz-Beech
May 14 2026

Abstract

Tactical combat AI in games and simulations is conventionally implemented using scripted approaches such as finite state machines and behaviour trees. These are deterministic, cannot learn, and fail to adapt to novel opponent strategies. Reinforcement learning addresses these limitations, but existing research operates at extremes, either industrial-scale projects requiring thousands of TPU cores (AlphaStar, OpenAI Five) or simple benchmark environments lacking tactical complexity. This dissertation addresses the gap between these extremes.

A Gymnasium-compatible 2D battle simulator was designed and implemented, featuring procedurally generated terrain with three terrain types and five height levels, three distinct unit types (infantry, cavalry, and archers) with differentiated combat mechanics including a shield wall ability, and partial observability enforced by forest concealment and Bresenham line-of-sight tracing. A RecurrentPPO agent with an LSTM-based policy was trained against this environment using a four-stage curriculum that progressively introduced terrain mechanics, combined with self-play against frozen policy snapshots. Training ran for approximately 100 million environment steps on consumer hardware (Ryzen 5 3600, RTX 3060) without cloud infrastructure.

The system was evaluated through 126 automated tests (96.7% pass rate), TensorBoard training metrics, and human acceptance testing. Explained variance reached 0.9986 and total loss decreased by three orders of magnitude over training. Human evaluation of agent behaviour produced a mean plausibility score of 4.0 out of 5. The trained agent exhibited emergent tactical behaviour not explicitly encoded in the reward function, including diagonal routing that produced flanking effects and the unprompted halting of archer formations on elevated terrain to exploit the height range bonus.

These results establish that RecurrentPPO with an LSTM policy can produce deliberate, tactically meaningful behaviour in a mid-scale combat environment using accessible hardware and open-source tooling. The primary limitation was a failure in the self-play pipeline, which prevented exposure to adaptive adversarial strategies and likely constrained the sophistication of multi-formation coordination.

Keywords: Reinforcement Learning, RecurrentPPO, LSTM, tactical

Report’s total word count: 17,395

Acknowledgements

I would like to thank the 3-time European Breachers champion, Ethan Rolfe for providing feedback in chapter 7, and my supervisor for providing feedback.

Contents

Introduction

1.1 Background

Tactical combat scenarios present a significant challenge for artificial intelligence in games and simulations. Unlike simpler decision-making contexts, tactical combat requires individual units to coordinate their actions, respond to enemy positioning, select appropriate formations, and operate under incomplete information about the opponent's intentions. The quality of the AI's decision-making in these scenarios directly affects whether engagements feel challenging and dynamic, or predictable and repetitive.

Conventional approaches to game AI rely on scripted methods such as finite state machines (FSMs) and behaviour trees. These approaches react to player actions, but their responses are deterministic, the same situation will always produce the same behaviour.

1.2 Problem Statement

While these techniques are well-established and intuitive to implement, they carry several fundamental limitations.

First, they are inherently static: a behaviour tree and finite state machine will produce the same response to the same situation every time; meaning an individual will have a similar experience each time leading to predictable gameplay loops that users can learn to exploit, also known as a “meta” (the most efficient way to play), which leads to these exploits being shared, removing the novelty of each individuals experience. These optimal exploit strategies are rapidly disseminated through online communities, further reducing the challenge for all players.

Second, they are entirely hand-authored, meaning that every possible tactical scenario must be anticipated and manually coded by the developer. As the complexity of the environment grows, more unit types, more terrain features, more possible enemy configurations, the number of rules required scales rapidly, and edge cases are inevitably missed.

Third, and most critically for tactical applications, scripted approaches cannot learn. An FSM or behaviour tree will never improve its performance through experience. If a user employs a novel strategy that the developer did not anticipate, the AI has no mechanism to adapt, resulting in ineffective or incoherent responses.

1.3 Alternatives

Reinforcement learning (RL) offers a fundamentally different approach that addresses each of these limitations. Rather than relying on pre-programmed rules, an RL agent learns by repeatedly interacting with an environment, receiving numerical rewards for desirable outcomes, such as winning an engagement or maintaining formation cohesion, and penalties for poor outcomes, such as losing units unnecessarily. Over thousands of training episodes, the agent's policy, its mapping from observations to actions, gradually improves through trial and error. This means the resulting tactical behaviour is not limited to scenarios the developer explicitly anticipated; the agent discovers effective strategies on its own, including responses to situations that would be difficult or impossible to hand-code.

However, much of the existing research on RL in games operates at one of two extremes. At the upper end, projects such as AlphaStar (Vinyals et al., 2019) and OpenAI Five (OpenAI, 2019)

have demonstrated superhuman performance in complex strategy games but require computational resources far beyond what is accessible to most researchers or developers. At the lower end, RL is frequently benchmarked on simple toy environments such as CartPole or grid worlds, which lack the complexity of real tactical decision-making. There is a notable gap in applying RL to mid-scale tactical combat, scenarios involving formation control, partial observability, and multi-unit coordination, using accessible tools and modest computational resources.

1.4 Aims and Objectives

This dissertation addresses that gap. It presents the design and evaluation of an adaptive AI framework in which RL agents learn to control formations and individual units in tactical engagements. The aim is to demonstrate that RL can produce adaptive, non-trivial tactical behaviour at a scale between toy benchmarks and industrial-grade systems, using a framework that is designed to be modular and environment-agnostic.

To achieve this aim, the project will pursue the following objectives:

- Design and implement a Gymnasium-compatible 2D battle environment featuring procedural terrain, partial observability, and multiple unit types to provide a sufficiently complex training environment.
- Train RL agents using RecurrentPPO with LSTM-based policies, alongside alternative algorithms, to learn adaptive tactical behaviour.
- Evaluate trained agents against scripted baselines using measurable metrics such as win rate and formation cohesion.
- Compare RecurrentPPO against alternative RL architectures to evaluate the hypothesis that PPO with LSTM-based recurrent policies is well-suited to this problem domain.
- Ensure the system is computationally lightweight enough to run inference without dedicated GPU hardware.
- Address ethical and legal considerations throughout the project.

1.5 Proposed Solution Approach

The proposed solution consists of the following components:

- Environment Design: A procedurally generated 2D battlefield with varying terrain heights and types, enabling the agent to learn terrain-based tactical advantages beyond basic survival behaviour.
- Unit Composition: Multiple troop types with distinct attributes, requiring the agent to learn differentiated offensive and defensive strategies.
- Model Training and Evaluation: A comparison of RL architectures with primary focus on RecurrentPPO with LSTM, evaluated against alternative approaches to validate the chosen method.
- System Architecture: A modular, Gymnasium-compatible design to facilitate reproducibility and future extension

Social, Legal and Ethical Considerations

Although this project is an academic exercise focused on a single player 2D battle simulator, the techniques it employs, reinforcement learning for tactical decision-making, formation control, and unit coordination under partial observability; intersect with broader societal, legal and ethical questions. This chapter addresses those considerations honestly, distinguishing between issues that directly affect the project and those that arise from the wider implications of the research domain.

2.1 Social Implications

The primary social consideration surrounding this project is its dual-use potential. While the system is designed as an academic framework for training AI agents in a simulated tactical environment, the underlying techniques are directly relevant to military applications. This is not a hypothetical concern. The United Kingdom's 2025 Strategic Defence Review placed artificial intelligence at the centre of future military capability, with the Ministry of Defence committing significant funding to autonomous systems and AI-driven targeting infrastructure (UK Ministry of Defence, 2025). Internationally, research into RL-based tactical decision-making is actively pursued by defence organisations, with projects such as AlphaStar and subsequent military-funded research demonstrating the transferability of game AI techniques to real-world combat scenarios.

The concern with autonomous tactical AI is well documented in academic literature. Scholars have argued that autonomous systems risk eroding moral restraint in warfare, creating environments in which technological processes replace human moral reasoning (Renic and Schwarz, 2025). When machines make or influence decisions about the application of force, questions of accountability become significantly more complex. The UK government's position is that accountability cannot be transferred to a machine and that human responsibility must be maintained regardless of the level of autonomy in a system (UK Government, 2024).

It should be stated clearly that this project does not develop a weapons system, nor is it intended for military application. The simulator operates on abstract 2D units with no depiction of graphic violence, and the research is conducted entirely within an academic context. Nevertheless, acknowledging the dual-use nature of the techniques employed is an important part of responsible research practice. The intent of this work is to explore the viability of RL for adaptive game AI, and any further application of these methods would require its own independent ethical review.

2.2 Legal Considerations

This project does not collect, store or process personal data of any kind. There is no user-facing deployment, no network connectivity requirement, and no data transmitted to external servers. As such, data protection legislation including the UK General Data Protection Regulation (UK GDPR) and the Data Protection Act 2018 do not apply directly to the system. The simulator is designed to operate entirely offline on local hardware.

All third-party libraries and frameworks used in the project are released under permissive open-source licences. Stable-Baselines3 and its contrib package are released under the MIT licence. Gymnasium, the environment interface standard, is also MIT-licensed. PyTorch, the underlying deep learning framework, is released under the BSD licence. All these licences permit

academic use, modification and redistribution. No proprietary datasets, copyrighted game assets or commercially restricted resources have been used in the development of this project.

While no specific legislation governs the development of AI for use in simulated game environments, it is worth noting the broader legal landscape. Internationally, discussions on the regulation of autonomous weapons systems are ongoing. A United Nations General Assembly resolution in 2023, supported by 164 member states, called for urgent action on lethal autonomous weapons systems, with a follow-up resolution in 2024 receiving support from 166 states (UN General Assembly, 2024). The House of Lords AI in Weapon Systems Committee published a report titled 'Proceed with Caution' in 2023, recommending that the UK government adopt an operational definition of autonomous weapons and lead international engagement on their regulation (House of Lords, 2023). Although these developments do not impose legal obligations on this project, they reflect the evolving regulatory context within which research on AI for tactical decision-making is situated.

2.3 Ethical Considerations

Subject matter. The project simulates tactical combat, a subject that some individuals may find objectionable on moral grounds. However, war and strategy games are a well-established genre with a long history in both entertainment and education, including use in professional military training. The simulator in this project uses abstract 2D representations with no depiction of injury, death or suffering. The focus is on the decision-making process of the AI agent rather than on the visual or narrative portrayal of conflict.

Emergent behaviour. A characteristic of reinforcement learning is that agents may discover strategies that were not anticipated by the developer. While this adaptability is the primary motivation for using RL in this project, it also raises the possibility of the agent learning degenerate or undesirable behaviours, for example, exploiting unintended mechanics in the environment rather than developing genuine tactical skill. This is a known challenge in RL research and is mitigated through careful reward function design and evaluation of trained agent behaviour. Any emergent strategies are documented and analysed in the evaluation chapter of this dissertation.

Training bias. The agent's behaviour is shaped entirely by the training environment. If the procedurally generated terrain and scenarios lack sufficient diversity, the agent may learn strategies that only perform well under narrow conditions, failing to generalise to novel situations. This is analogous to the dataset bias problem in supervised learning. The project addresses this by incorporating procedural generation with varying terrain types, heights and unit compositions to expose the agent to a broad range of tactical situations during training.

Computational impact. Training reinforcement learning agents is computationally intensive and carries an associated energy cost. Large-scale RL projects such as AlphaStar and OpenAI Five required substantial data centre resources over extended periods. A deliberate design goal of this project is to demonstrate that adaptive tactical AI can be achieved with modest computational resources. All training is conducted locally on consumer-grade hardware without reliance on cloud computing infrastructure, significantly reducing the environmental footprint compared to industrial-scale RL research.

Use of AI tools. In accordance with the University of Reading's policy on the use of artificial intelligence tools in assessed work, AI-assisted writing tools were used during the preparation of this dissertation as an editorial and structuring aid. All graphs were generated by AI, to

efficiently convert the authors drawn graphs into readable digital graphs. All ideas, arguments and technical content originate from the author's own work. The use of AI tools is disclosed here in the interest of academic transparency.

2.4 Summary

This chapter has examined the social, legal and ethical dimensions of developing a reinforcement learning framework for tactical combat AI. Socially, the project acknowledges the dual-use potential of its techniques while clearly situating itself within an academic and educational context. Legally, the project operates within the terms of open-source licensing and does not engage with personal data, though it notes the evolving international regulatory landscape around AI in combat. Ethically, the project addresses the subject matter, the possibility of emergent agent behaviours, training bias, computational impact and the transparent use of AI writing tools. These considerations inform the design and evaluation choices discussed in subsequent chapters

Literature Review

3.1 Background

This literature review examines the current landscape of artificial intelligence for tactical decision-making in games and mid-scale simulations. It traces a progression from conventional scripted approaches up to modern reinforcement learning techniques, identifying the strengths and limitations of each and establishing the gap that this project addresses. The review is structured as follows: Section 3.2 examines traditional game AI techniques and their limitations; Section 3.3 introduces reinforcement learning as an alternative paradigm and discusses the specific architectural choices of PPO and LSTM; Section 3.4 surveys existing applications of RL to combat and strategy games; and Section 3.5 summarises the literature review.

3.2 Traditional Game AI Approaches

3.2.1 Finite State Machines

Finite state machines (FSMs) are one of the oldest and most widely used techniques for controlling non-player character behaviour in games. An FSM consists of a fixed set of states, a set of transitions between those states, and a set of conditions that trigger each transition. At any given moment, the agent occupies exactly one state and executes the behaviour associated with it. When a condition is met, the agent transitions to a new state and adopts the corresponding behaviour (Millington and Funge, 2009). Classic examples include the ghosts in Pac-Man, which transition between roaming, chasing, and fleeing states depending on the player's actions.

FSMs are attractive because they are simple to implement, easy to debug, and computationally inexpensive. However, there are several issues as discussed in Section 1.2. First, these approaches are entirely deterministic, meaning they are never unique as the agent will always act the same way given a stimulus, producing identical behaviour with each interaction (Millington and Funge, 2009). Second, scalability becomes an issue. This scalability problem is well-documented; Isla (2005) encountered it during the development of Halo 2, where FSMs with hundreds of states became unmanageable. Third, FSMs cannot learn, as they are made of a pre-determined set of states, these will never change (Millington and Funge, 2009).

3.2.2 Behaviour Trees

Behaviour trees emerged as a response to the scalability problems of FSMs. Rather than defining explicit transitions between every pair of states, a behaviour tree organises decisions into a hierarchical tree structure. At each update, the tree is traversed from the root, evaluating conditions at branch nodes and executing actions at leaf nodes. This structure makes behaviour trees more modular than FSMs: individual branches can be added, removed, or modified without affecting the rest of the tree (Colledanchise and Ogren, 2018). Behaviour trees gained widespread adoption following their use in Halo 2 and have since become the dominant approach to game AI in commercial titles.

Despite their improved modularity, behaviour trees share the same fundamental limitations that were discussed in Section 1.2 and Section 3.2.1 (FSMs). Most importantly, like FSMs, behaviour trees cannot learn or adapt. They encode fixed decision logic that does not improve through

experience (Colledanchise and Ogren, 2018). For tactical combat scenarios, where the space of situations is vast and opponents may employ novel strategies, this inability to adapt represents a significant limitation.

3.3 Reinforcement Learning

Reinforcement learning (RL) is a type of machine learning in which an agent learns to make decisions by interacting with an environment and receiving feedback in the form of numerical rewards. The agent observes the current state of the environment, selects an action, receives a reward signal indicating how good or bad the outcome was, and transitions to a new state. Over many iterations of interaction, the agent's policy, its mapping from observations to actions, is improved to maximise its cumulative reward (Sutton and Barto, 2018). Unlike the limitations of scripted approaches stated in Sections 1.2 and 3.2, RL agents are not limited to developer-anticipated scenarios; they discover effective strategies through trial and error, including responses to situations that would be difficult or impossible to hand-code.

The fact that RL agents are not limited to pre-defined responses means they can react to unanticipated stimuli. Recent research has demonstrated this; in a 2025 study of multi-agent pursuit-evasion games, agents trained with multi-agent reinforcement learning developed novel cooperative tactics, including asymmetric role allocation where one pursuer minimised its own effort while complementing the actions of another, a strategy not thought of by the researchers (Li et al., 2025). This capacity for emergent behaviour discovery is precisely what is needed in tactical combat, where the space of situations is too large to enumerate manually.

3.3.1 Proximal Policy Optimization

Among RL algorithms, Proximal Policy Optimization (PPO) has become one of the most widely adopted methods for training agents in complex environments. PPO belongs to the family of policy gradient methods, which directly optimise the agent's policy rather than learning a value function from which actions are derived. The central challenge in policy gradient methods is controlling the size of each policy update. If updates are too small, learning is slow; if they are too large, the policy can change drastically in a single step, destroying previously learned behaviour and preventing recovery (Schulman et al., 2017).

PPO addresses this challenge through a clipped surrogate objective function that limits the magnitude of each policy update. By capping how much the policy can change in a single optimisation step, PPO ensures stable, incremental improvement without the risk of catastrophic updates. This makes it simpler to implement than its predecessor, Trust Region Policy Optimization (TRPO), while achieving comparable or superior performance across a range of benchmark tasks including simulated robotics and Atari game playing (Schulman et al., 2017). PPO's combination of stability, simplicity, and empirical effectiveness has made it the default choice for many game AI and robotics applications, including OpenAI Five for Dota 2 (OpenAI, 2019) and numerous combat simulation projects.

Alternative RL algorithms exist and warrant brief consideration. Deep Q-Networks (DQN) learn a value function that estimates the expected reward for each action in each state, then select the highest-value action. While effective in environments with discrete action spaces, DQN struggles with the continuous or high-dimensional action spaces common in tactical scenarios. Soft Actor-Critic (SAC) is an off-policy algorithm that maximises both reward and action entropy, encouraging exploration. SAC can be more sample-efficient than PPO but is more complex to implement and typically used for continuous control tasks such as robotics (de la Fuente and

Vidal Guerra, 2024). For the discrete, multi-unit action space of a tactical battle simulator, PPO's stability and proven track record in game environments make it the most appropriate choice.

3.3.2 Long Short-Term Memory Networks

A standard RL agent using a feedforward neural network makes each decision based solely on the current observation. It has no memory of previous observations and no capacity for temporal reasoning. In fully observable environments, where the current state contains all information needed for optimal decision-making, this is sufficient. However, many real-world and game environments are partially observable, the agent cannot see the entire state of the world at any given moment. In tactical combat, units may be obscured by terrain, enemy movements may only be visible intermittently, and the significance of the current situation depends on what happened earlier in the engagement.

Long Short-Term Memory (LSTM) networks address this limitation by maintaining a hidden state that persists across time steps, allowing the network to retain and integrate information from previous observations. LSTM was originally introduced by Hochreiter and Schmidhuber (1997) as a solution to the vanishing gradient problem in recurrent neural networks, enabling the learning of long-range temporal dependencies that previous recurrent architectures could not capture. The architecture uses a gating mechanism, input, forget, and output gates, to control what information is stored, discarded, or passed forward at each time step.

The application of LSTM to reinforcement learning under partial observability was demonstrated by Hausknecht and Stone (2015), who replaced the fully connected layer of a Deep Q-Network with an LSTM, creating a Deep Recurrent Q-Network (DRQN). Their experiments showed that DRQN successfully integrated information across time steps even when the agent could only observe a single frame at each decision point. Crucially, when trained with partial observations and evaluated with varying levels of observability, DRQN's performance scaled with the amount of information available, demonstrating that the recurrent architecture genuinely learned to use temporal information rather than simply memorising state patterns (Hausknecht and Stone, 2015). This finding directly supports the use of LSTM-based policies in the present project, where units operate under partial observability due to terrain-based line-of-sight restrictions.

While Hausknecht and Stone demonstrated the effectiveness of LSTM within a value-based framework (DQN), their findings regarding temporal integration under partial observability are architecture-agnostic, the benefit of recurrent memory applies regardless of the underlying RL algorithm. This project therefore combines LSTM with PPO rather than DQN, leveraging the training stability of PPO discussed in Section 3.3.1 with the temporal reasoning capabilities demonstrated by Hausknecht and Stone.

3.3.3 Combining PPO with LSTM

The combination of PPO's stable policy optimisation with LSTM's temporal memory is implemented in the Stable-Baselines3 Contrib library as RecurrentPPO (Raffin et al., 2021). This implementation integrates an LSTM layer into the policy network, allowing the agent to maintain a hidden state across sequential observations while benefiting from PPO's clipped updates during training. The Stable-Baselines3 library provides reliable, well-tested implementations of standard RL algorithms with a consistent API built on top of PyTorch, making it suitable for academic research where reproducibility is important. RecurrentPPO is the primary training algorithm used in this project.

3.4 Reinforcement Learning in Combat and Strategy Games

Reinforcement learning has been applied to combat and strategy games at a range of scales, from simple grid-based environments to full commercial titles. Many of these applications employ curriculum learning, a technique in which the agent is initially exposed to simpler aspects of the environment, with complexity gradually increasing as the agent's competence improves (Bengio et al., 2009). This approach mitigates the problem of sparse rewards, where feedback is so infrequent that the agent struggles to learn any meaningful behaviour. Examining this existing work reveals both the potential of RL for tactical AI and the gap that the present project addresses.

3.4.1 Large-Scale Applications

At the upper end of complexity, DeepMind's AlphaStar achieved Grandmaster level in StarCraft II, placing within the top 0.2% of human players on the official European ladder. The system used a combination of imitation learning from human replays and multi-agent reinforcement learning, training a population of agents that played against each other to develop diverse and robust strategies (Vinyals et al., 2019). AlphaStar demonstrated that RL can handle the enormous state and action spaces of a real-time strategy game, including resource management, unit production, and tactical micromanagement. However, the computational cost of training was substantial, requiring thousands of TPU cores over multiple weeks, resources far beyond what is available to most researchers or game developers.

Similarly, OpenAI Five achieved competitive performance against professional human teams in Dota 2 using PPO with self-play, demonstrating that PPO can scale to complex multi-agent strategy games (OpenAI, 2019). Like AlphaStar, the computational requirements were immense. These projects serve as existence proofs that RL can produce superhuman tactical behaviour, but their resource requirements make them impractical as models for accessible game AI development.

This project does not have access to such resources, and the goal of the project is not to beat every human, but to demonstrate that adaptive tactical AI can be achieved with modest computational resources.

3.4.2 Small-Scale Environments

At the other end of the spectrum, much RL research is conducted in simple benchmark environments such as those provided by Gymnasium (Brockman et al., 2016). Environments like CartPole, MountainCar, and LunarLander are valuable for testing algorithmic properties but lack the complexity of real tactical decision-making; they involve single agents, small state spaces, and no coordination or formation control. While essential for foundational RL research, results in these environments do not straightforwardly transfer to the challenges of multi-unit tactical combat.

Between these extremes, environments such as microRTS (Ontanon et al., 2018) and Deep RTS (Andersen et al., 2018) have been developed specifically for RL research in strategy games. Andersen et al. (2018) explicitly noted that few game environments offer complexity suited to current RL research, and that there is a lack of flexibility in existing solutions.

Deep RTS was designed to bridge the gap between the simplicity of microRTS and the complexity of StarCraft II, offering configurable difficulty levels for planning, reasoning, and control. Whilst Deep RTS is a good showcase of what can be achieved for AI opponents, it does

not satisfy this project's requirements as Deep RTS encompasses the full scope of real-time strategy gameplay including resource gathering, base construction, and unit production. The present project focuses specifically on the tactical combat layer in isolation: formation control, unit positioning, and engagement decisions; without the strategic elements of resource management.

These mid-complexity environments are the most relevant comparisons for the present project.

3.4.3 Tactical Combat with RL

Research applying RL specifically to tactical combat, as opposed to full RTS game-playing, remains limited. Boron et al. (2020) applied PPO to develop combat behaviour in a military constructive simulation, training agents in company-level engagements involving two-versus-one and two-versus-two force configurations. Their agents learned behaviours consistent with tactical principles such as massing forces, demonstrating that PPO can produce meaningful tactical behaviour even in simple combat environments. However, their work did not incorporate partial observability or formation control.

More recently, multi-agent RL has been applied to combat-themed games with promising results. A 2025 study compared DQN, PPO, and QMIX for training fighting NPCs in a role-playing game across configurations ranging from one-versus-one to seven-versus-seven engagements. PPO-trained agents achieved the highest user satisfaction ratings in most configurations, and the resulting NPCs defeated human players in all battle scenarios (Garcia et al., 2025). This provides empirical evidence that PPO is well-suited to combat AI, though the study focused on individual character fighting rather than formation-level tactics.

3.4.4 Identified Gap

The existing literature reveals a clear gap. Large-scale projects like AlphaStar and OpenAI Five demonstrate that RL can produce sophisticated tactical behaviour but require computational resources that are inaccessible to most developers. Small-scale benchmark environments lack the complexity of tactical combat. Mid-scale RL environments like Deep RTS provide better complexity but do not focus specifically on formation-based tactical combat with partial observability. Existing combat-focused RL research has demonstrated the viability of PPO for fighting AI but has not combined formation control, terrain-based partial observability, and multiple unit types within a single accessible framework. This project addresses that gap.

3.5 Summary

This literature review has traced the evolution of game AI from scripted approaches through to modern reinforcement learning methods, establishing the rationale for the architectural decisions made in this project. Traditional approaches, finite state machines and behaviour trees, are well-established but fundamentally limited by their inability to learn, adapt, or handle situations not anticipated by the developer. Reinforcement learning addresses these limitations by enabling agents to learn effective strategies through interaction with an environment. PPO provides stable, incremental policy improvement suitable for complex game environments, while LSTM networks equip the agent with temporal memory necessary for decision-making under partial observability. Existing applications of RL to strategy and combat games demonstrate the viability of these techniques but reveal a gap in mid-scale tactical combat involving formations, terrain, and multiple unit types. This project fills that gap by combining

RecurrentPPO with a custom Gymnasium-compatible tactical battle environment designed to be accessible, modular, and environment-agnostic.

Methodology

4.1 Requirement analysis

The system is a 2D battle simulator in which two AI agents command opposing armies across a procedurally generated battlefield. The terrain must have varying terrain types and height variations, affecting an agent's observation space and attributes.

Each agent should command up to six formations of variable size, composed of one of three-unit types. These should be Archers, Cavalry, and Infantries.

At each timestep, the agent should output a continuous action vector per formation.

The environment must be capable of simulating an arbitrary number of units and formations and rendering them through a visualiser that displays real-time battle relevant diagnostic information including per-formation cohesion, hit points, and kill rates.

Given the computational cost of training in a mechanically complex environment, the primary success criterion is whether the agent demonstrates a capacity to learn; specifically, whether it exhibits deliberate tactical behaviour capable of winning engagements, rather than converging on a fixed numerical benchmark. The objective is to validate that the methodology is viable for similar use cases, rather than to achieve state-of-the-art performance.

Training was conducted on a Ryzen 5 3600 with an RTX 3060, requiring approximately 70 hours for 100 million environment steps.

The software stack comprised Python 3.11.9, Stable-Baselines3 2.8.0, SB3-Contrib 2.8.0, NumPy 2.3.3, SciPy 1.16.2, and Gymnasium 1.2.3, pytest, and unittest.

4.2 Development Methodology

The project initially followed a waterfall-inspired approach in which each component, environment mechanics, observation space, training pipeline, was designed, implemented, and tested in sequence before moving to the next. However, as the system grew in complexity, this proved increasingly inefficient. Changes to one component, such as the observation space or core combat logic, had cascading effects that rendered sequential development impractical.

The methodology was therefore adapted to an iterative approach in which features were implemented in parallel, tested holistically, and refined based on the outcomes of training runs. While this meant the model had to be retrained whenever the observation space or environment logic changed significantly, it provided a clearer feedback loop: each training run could be evaluated against previous runs, making it possible to attribute improvements or regressions to specific changes. The way learning milestones where checked was a mixture of informal eyewitness evaluation and comparing win rates.

Version control was managed through GitHub, and GitHub Copilot was used as a development aid during implementation.

4.3 Training Workflow

The training pipeline follows the standard Gymnasium interaction loop. At each timestep the environment constructs an observation vector for every formation visible to the agent,

consisting of: presence (alive or dead), unit type, relative strength (proportion of living units), spatial bounds (minimum x, maximum x, average y, normalised to world size), average terrain height and type at the formation's position, shield wall status, and formation width and rank count normalised to the world diagonal. The agent's policy then returns an action vector per formation as described in Section 4.1.

The policy is updated based on a composite reward signal with four components. First, an HP differential term rewards damage dealt to enemy formations and penalises damage sustained. Second, an approach reward encourages closing distance to enemy formations. Third, a cohesion term rewards maintaining tight formation structure. Fourth, terminal bonuses apply a large positive reward for victory and a penalty for defeat. Upon episode termination or truncation, the environment resets with a regenerated terrain map and newly selected formation compositions.

Training uses a curriculum staged across four levels of environment complexity, each of which is trained first against a scripted opponent and then against snapshots of the agent's own policy via self-play. The stages progressively introduce environment mechanics: Stage 0 enables basic movement only, Stage 1 adds terrain-based speed modifiers, Stage 2 introduces forest concealment, and Stage 3 adds height and line-of-sight mechanics. The agent's learned weights carry over between stages, allowing knowledge from simpler environments to transfer into more complex ones. Self-play opponents are created by snapshotting the current policy once the agent's win rate exceeds a threshold, ensuring the agent trains against progressively stronger versions of itself.

4.4 Choice of Technologies

The selection of PPO as the training algorithm and LSTM as the policy architecture is justified in Sections 3.3.1–3.3.3. PPO provides training stability through its clipped surrogate objective, while LSTM equips the agent with temporal memory necessary for decision-making under the partial observability introduced by terrain and line-of-sight mechanics. The combination is implemented as RecurrentPPO via the Stable-Baselines3 Contrib library (Raffin et al., 2021).

Stable-Baselines3 was selected over alternatives such as RLlib, CleanRL, and a custom implementation based on prior experience and ease of use. SB3 provides a consistent, well-documented API built on PyTorch with reliable implementations of standard RL algorithms, making it suitable for a project where the primary research contribution lies in the environment design and training methodology rather than in novel algorithm development.

For combat targeting determining which enemy units are within attack range of each unit two approaches were considered. A naive nested loop checking every unit against every other unit has $O(n^2)$ time complexity per step. A cKDTree, a k-dimensional binary space-partitioning tree implemented in C++ and wrapped in Cython as part of SciPy's spatial module, reduces nearest-neighbour queries to O(log n) on average for typical spatial distributions. In a k-d tree, each node partitions space along alternating axes, enabling efficient spatial queries by pruning large regions of the search space. Given that this distance check is performed every environment step for every living unit, the performance difference was substantial, and the cKDTree was adopted accordingly.

The visualisation layer, built with tkinter and matplotlib, was selected based on familiarity rather than comparative evaluation. The focus of this project is the viability of RecurrentPPO for mid-

scale tactical combat; the visualiser serves as a diagnostic and demonstration tool rather than a research contribution, and the choice of visual library does not affect training or evaluation.

4.5 Evaluation Metrics

The objective of this project is to establish whether RecurrentPPO with an LSTM-based policy can learn deliberate, tactically meaningful behaviour in a mid-scale combat environment, not to demonstrate optimality or state-of-the-art performance. Evaluation is therefore structured around three concrete criteria.

First, behavioural improvement over training: a later training checkpoint should achieve a higher win rate and mean reward against the scripted baseline than an earlier one, as measured by Section 7.1.22's comparative evaluation across multiple episodes. This provides a quantitative signal that the agent's policy improved rather than stagnated.

Second, reward signal validity: the reward function must produce finite, bounded, and directionally correct signals throughout training. This is verified by Section 7.1.18, which confirms terminal rewards respond correctly to victory and defeat, and by Section 7.3.5, which confirms no NaN or out-of-bounds values occur across full episodes at all curriculum stages.

Third, behavioural plausibility: the agent's decisions must appear tactically coherent to an independent human observer with no prior knowledge of the system, as assessed by AT-1's rubric evaluation across five episodes. A mean score of 3.0 or above on a five-point scale, where 3 represents consistent engagement with the enemy and reactive behaviour, constitutes a pass.

The test suite additionally provides supporting verification through non-functional testing, which confirms memory stability, simulation determinism, graceful degradation under extreme unit counts, renderer frame rate, and observation numerical stability. Acceptance testing further validates that curriculum progression is visible to an independent observer, the visualiser conveys battlefield state intelligibly, and the training pipeline is reproducible from the described configuration.

Together these criteria establish viability: that the training pipeline produces a measurable improvement in policy quality, that the reward structure functions correctly, and that the resulting behaviour is recognisable as intentional rather than random.

Design

This system contains several components, the model, the controller, the view, the training environment, training script, and the curriculum and self-play call back. I will discuss the design of the model, training environment, training script, and the curriculum and self-play call back. The view and the controller will not be discussed in detail as mentioned in Section 4.4, as it is irrelevant to this paper and is purely to visualize the agents.

5.1 Model

The model contains all logic of the simulation and is the most crucial part of the simulation.

5.1.1 Features

As mentioned in Section 4.1 the model contains varying terrain types and heights, and contains Archers, Cavalry, and Infantries. The three terrain types are grass, which is just standard terrain with no buffs or nerfs, water, which reduces speed and is always at a height of one, and forest, which conceals units and has a slight speed nerf. The heights can vary from 1-5, a higher height makes archers have further range, and if there is a peak between two units, they are concealed to each other.

Archers are a very weak unit however have significant range, for computational efficiency the units will use hit registering instead of an actual simulated arrow. Cavalries are a fast version of infantry, with significantly more health but reduced damage. Infantries are the all-round unit; in most games infantries are used as an allrounder cheap unit used to fill in ranks.

However, this project is for demonstrating RL capabilities not in game economics. Therefore, I made it so infantry could use a special ability called “shield wall,” which reduces incoming and outgoing damage, reduces speed, and unit spacing.

5.1.2 Classes

The simulation is built around a three-tier hierarchy of Team, Formation, and Unit. A Team represents one side of the battle and owns a collection of Formations. It is responsible for aggregating information across its formations, such as retrieving all living units or determining whether the team has been defeated. Each agent controls exactly one Team.

A Formation is a group of Units that move and fight as a coordinated body. It receives movement commands from the agent in the form of port and starboard coordinates, which define a line along which its units arrange themselves in ranks. The Formation tracks its own structural properties and is the level at which the agent reasons and issues orders. Formations do not interact with each other directly; coordination between formations is the agent's responsibility.

A Unit is the atomic element of the simulation. Each unit has a position, hit points, damage output, attack range, and movement speed. Units belong to exactly one Formation and act on destinations assigned by that Formation; they have no autonomous decision-making.

The World encapsulates the battlefield itself. It owns the terrain map, height map, and all Teams. It is responsible for the per-tick simulation loop. The World also manages visibility and line-of-sight calculations, determining which enemy formations are included in each agent's observation space. Terrain generation, army population, and curriculum-gated mechanics are all managed at this level.

5.1.3 Tick logic

Each simulation step is structured as a tick, executed in a fixed sequence of three phases. First, collision resolution pushes overlapping units apart to prevent formations from collapsing into a single point. Second, the combat phase determines which units are in range of an enemy and executes attacks; Units that can see an enemy but cannot reach it will instead move toward that target. Third, the movement phase advances all non-attacking units toward their assigned destinations, with movement speed modified by the terrain beneath them. This ordering is deliberate: resolving collisions before combat prevents units from stacking to concentrate damage unfairly, and running combat before movement ensures that an attacking unit commits to its engagement for that tick rather than repositioning and striking simultaneously.

5.2 Training

The main logic of the training is managed by two separate files, the training environment and the training set up. The training environment contains training logic and the training set up contains parameters for the algorithm.

5.2.1 Training environment

The training environment wraps the simulation as a Gymnasium-compatible environment, exposing a continuous action space and a flat observation vector to the agent. Each agent controls up to six formations, producing a flat action vector of thirty values: Five per formation slot. The first four values per slot define the port and starboard coordinates normalised to the range [-1, 1], which are rescaled to world coordinates at each step. The fifth value is a shield wall toggle, activated when positive. If the agent attempts to activate shield wall on a non-infantry formation, a penalty is applied to discourage learning this as a viable action.

The observation space is a flat vector of 171 values. The first eighty-four values encode the agent's own formations (up to six slots of fourteen features each), the next eighty-four encode enemy formations in the same format, and the final three values encode global state: the current timestep as a proportion of the episode limit, and the world dimensions. Enemy formations that are not visible due to concealment or broken line of sight are zeroed out, presenting the agent with a partially observable environment. For self-play, the opponent receives a mirrored observation where the x-axis is flipped, ensuring both agents perceive the battlefield from an equivalent perspective regardless of which side they spawn on.

The reward function combines four components. An HP differential term rewards damage dealt to the enemy and penalises damage sustained, scaled relative to total army health to remain meaningful as armies shrink. An approach term rewards reducing the average distance between friendly and enemy formation centres. A cohesion term rewards formations that maintain tight structure, with peak reward at 0.7 cohesion and a penalty for formations that stretch beyond twice their expected length. Terminal bonuses provide a large positive reward for victory and a penalty for defeat. Frame skipping allows the simulation to advance multiple ticks per agent decision, reducing the effective decision frequency without altering the underlying simulation fidelity.

At each step, the observation space is constructed and passed to the agent, which returns an action vector. The environment then applies the action, advances the simulation, and returns the resulting observation alongside the computed reward. Upon termination or truncation, the environment resets: the terrain is regenerated and new formation compositions are selected

randomly, ensuring the agent trains across a diverse range of battlefield configurations rather than memorising a single layout.

5.2.2 Training Configuration

Training is orchestrated through a separate script that defines the algorithm, parallelisation strategy, and curriculum progression. The agent is trained using RecurrentPPO from the SB3-Contrib library, which combines PPO's clipped policy updates with an LSTM layer that maintains a hidden state across sequential observations. Training is parallelised across five environments using SubprocVecEnv, which runs each environment in a separate process to maximise throughput on the available hardware.

The curriculum and self-play schedule are managed by a callback that monitors the agent's win rate over a rolling window. When the win rate exceeds a promotion threshold of 0.7 over the most recent one hundred episodes, the callback snapshots the current policy as a new self-play opponent and, where applicable, advances the curriculum stage. Checkpoints are saved at regular intervals to allow training to be resumed and to preserve intermediate policies for evaluation. The learning rate is set to 5e-5, and training runs are configured to continue from previous checkpoints with continuous timestep counting to maintain consistent logging across sessions.

5.3 Self play and curriculum learning.

The training process follows a staged curriculum that progressively introduces environment complexity, combined with self-play to ensure the agent is evaluated against an adaptive opponent at each stage. The progression operates as a state machine with two phases per curriculum stage: the agent first trains against a scripted opponent, then against a snapshot of its own policy.

5.3.1 Self Play

The scripted opponent follows a simple advancing strategy, moving all formations toward the agent at a fixed speed. Once the agent achieves a win rate above the promotion threshold over a rolling window of recent episodes, its current policy is saved as a frozen snapshot. The environment then switches to self-play, where this snapshot acts as the opponent, receiving a mirrored observation and issuing its own actions independently. When the agent again exceeds the promotion threshold against its own snapshot, the curriculum advances to the next stage, the opponent reverts to the scripted baseline, and the cycle repeats.

5.3.2 Curriculum learning

The four stages gate the introduction of environment mechanics in order of complexity. Stage 0 permits only basic movement and combat. Stage 1 activates terrain-based speed modifiers, requiring the agent to account for water and forest tiles when manoeuvring. Stage 2 introduces forest concealment, which removes formations hidden in woodland from the opponent's observation space. Stage 3 enables the full height system, including range bonuses for elevated positions and line-of-sight occlusion by terrain peaks. By the end of the curriculum, the agent has demonstrated the ability to win against an adaptive opponent in the fully featured environment.

Implementation

This chapter describes the low-level mechanics that realise the architecture set out in Chapter 5. The chapter is organised around four concerns: the world model and the classes that hold its state (Section 6.1), the procedural generation of the environment in which battles take place (Section 6.2), the per-tick simulation pipeline that advances that state (Section 6.3), and the reinforcement-learning interface that exposes the simulation as a Gymnasium environment with self-play support (Section 6.4). A summary closes the chapter (Section 6.5). Code listings for every section are collected in the appendix to keep the chapter within the dissertation word limit.

6.1 World Model

The world model is built around a three-tier class hierarchy, a set of module-level constants that govern simulation balance, and a population routine that instantiates a fresh battlefield at the start of each episode.

6.1.1 Unit Hierarchy

The unit system is implemented using an abstract base class with three concrete specialisations. The base Unit class defines the shared interface: position, hit points, damage, range, speed, and a movement method that advances the unit toward its destination by a distance proportional to its speed and the current tick rate (Snippet 1). Each unit also carries an is_attacking flag that is cleared at the start of every combat phase and set when the unit lands an attack.

Infantry override both attack and move. When shield wall is active, damage output is halved, movement speed is halved, and the formation tightens its spacing (Snippet 2). Archers override attack to apply a 95% damage reduction against infantry with shield wall active, representing the defensive effectiveness of a shield formation against ranged fire (Snippet 3). Cavalries override the can_move_while_attacking property to return True, making them the only unit type that continues moving during combat (Snippet 4). The class structure and the stat differences are shown in Figure 6.1 and tabulated below.

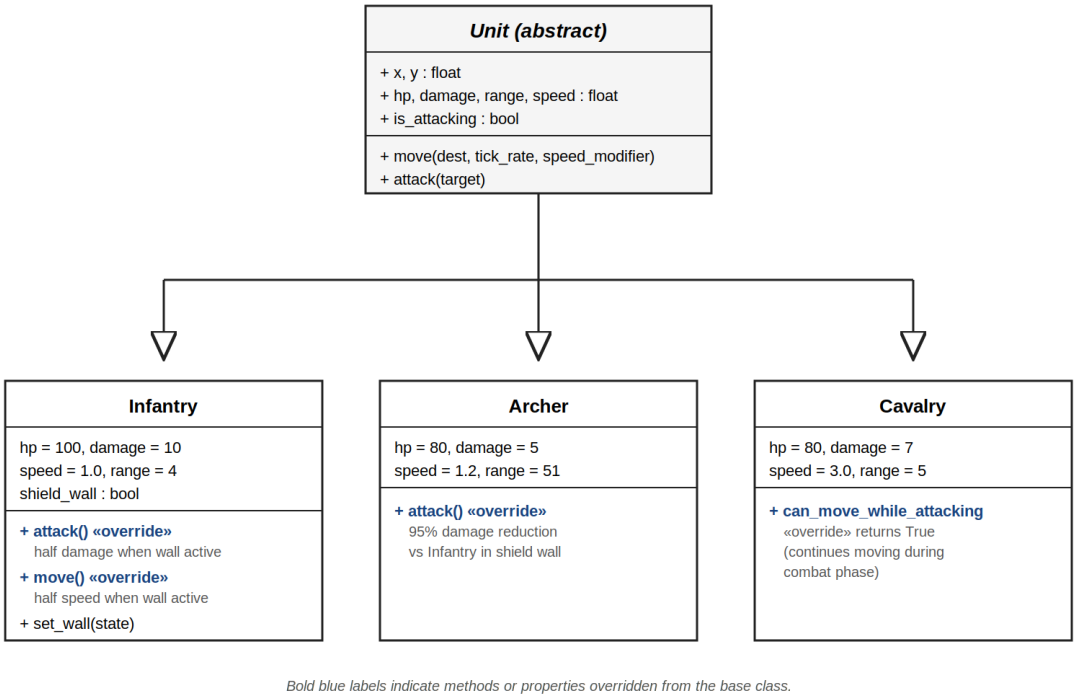


Figure 6.1: Unit class hierarchy. Bold blue labels mark methods or properties overridden from the base class.

Unit	HP	Damage	Speed	Range	Special
Infantry	100	10	1.0	4	Shield wall
Archer	80	5	1.2	51	Ranged; reduced damage vs shield wall
Cavalry	80	7	3.0	5	Attacks while moving

Table 6.1: Unit statistics.

6.1.2 Formation

A Formation receives port and starboard coordinates and distributes its units along the line between them. The line's length is divided by the unit slot size (twice the unit radius plus a gap) to determine how many units fit per rank. Remaining units are placed in subsequent ranks, offset perpendicular to the line by a fixed rank spacing. Each unit is assigned a destination at its calculated position; actual movement occurs during the tick's movement phase (Snippet 5).

When shield wall is active, both the unit gap and rank spacing are reduced by one, compressing the formation into a tighter block. This is applied at the formation level before destination calculation, so the compression is immediate and uniform across all units.

6.1.3 Team

The Team class is a thin aggregator that owns a list of Formations. It provides convenience methods to retrieve all units, all living units, and to check whether the team is defeated (no living units remain). No logic beyond aggregation exists at this level (Snippet 6).

6.1.4 Constants

The simulation's balance and behaviour are governed by a set of constants defined at module level. Terrain speed modifiers are defined as a dictionary mapping terrain type to multipliers: plains at 1.0, water at 0.3, and forest at 0.7. Forest detection range is set to 50 units, meaning concealed units in forest can still be detected if an observer is within this radius. The sight range is 100 units. Height range bonus is one additional range unit per height level difference. Default rank spacing and unit gap are both 2.0. Army compositions are defined as a list of presets, each specifying a name and a list of formation types and counts, from which the environment randomly selects during reset (Snippet 7).

6.1.5 Army Composition and Team Population

When a team is populated, the environment randomly selects one of ten army presets and constructs its formations accordingly. Each preset defines a list of formation specifications with a unit type and count. The formations are distributed vertically across the map with equal spacing and positioned horizontally near their respective edge of the battlefield, the friendly team spawns on the left and the enemy team on the right. Units within each formation are assigned initial positions by invoking the formation's movement method with the spawn coordinates, placing them along the port-starboard line immediately rather than requiring a tick to settle (Snippet 8).

6.2 Environment Generation

Each episode begins with a freshly generated battlefield. Terrain and elevation are produced procedurally, and a line-of-sight routine resolves which formations are visible to which observer based on the resulting height map.

6.2.1 Terrain Generation

The terrain map is generated in two stages. First, a Perlin noise function produces a continuous noise field over the world grid, built from six octaves of interpolated smooth noise with decreasing amplitude and increasing frequency (Snippet 10). The noise values are then thresholded into discrete terrain types: values below 0.15 become water, values above 0.85 become forest, and everything between becomes plains. Second, a cellular automata pass is applied for five iterations to smooth the terrain into contiguous regions, eliminating isolated single-tile patches (Snippet 11). The height map is generated using the same Perlin noise pipeline with different parameters (four octaves, scale of 25) and is quantised to integer values between 0 and 5. Water tiles are forced to height zero after generation.

6.2.2 Line of Sight

Line of sight between two units is determined by tracing a path across the height map using Bresenham's line algorithm (Snippet 15). Starting from the grid cell of the observer and ending at the grid cell of the target, each intermediate cell's height is compared against the maximum height of the two endpoints. If any intermediate cell exceeds this value, the line of sight is

blocked, and the target is excluded from the observer's observation space. This check is only active from curriculum stage 3 onward.

6.3 Per-Tick Simulation

Each simulation tick executes a fixed three-phase pipeline: collision resolution, combat targeting, and movement. The ordering, justified in Section 5.1.3, ensures that overlapping units are separated before combat is resolved, and that an attacking unit commits to its engagement for the tick rather than repositioning mid-strike. Figure 6.2 summarises the pipeline; the movement phase itself reuses the per-unit move method already described in Section 6.1.1 and is not given a separate subsection.

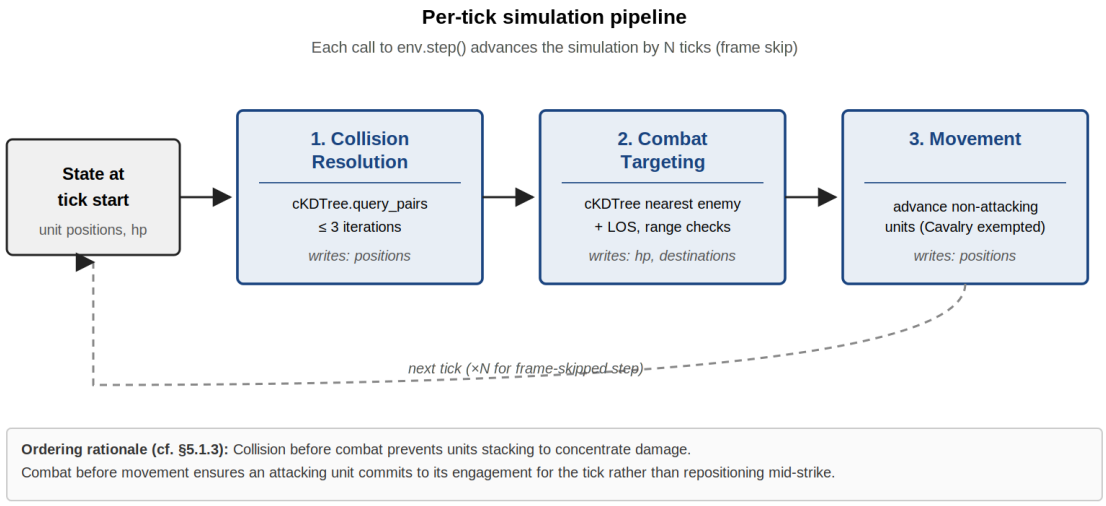


Figure 6.2: Per-tick simulation pipeline. Each call to env.step() advances the world by N ticks (frame skip).

6.3.1 Collision Resolution

Overlapping units are separated over three iterations per tick. A cKDTree is constructed from all living unit positions, and query_pairs returns all pairs within twice the maximum unit radius. For each pair where the actual distance is less than the sum of their radii, a separation vector pushes both units apart by half the overlap distance. Units are clamped to world bounds after each iteration. The process terminates early if no overlaps are found (Snippet 14).

6.3.2 Combat Targeting

Each tick, the combat phase builds a cKDTree from the positions of all living enemy units for each team. Every living attacker queries the tree for its nearest enemy. If the nearest enemy is within the attacker's effective range and passes the visibility checks, the attacker executes its attack method against that target. If the enemy is within sight range but out of attack range, the attacker's destination is updated to the enemy's position, causing it to advance toward the target during the movement phase (Snippet 13). Effective range accounts for height differential when curriculum stage 3 is active, adding one range unit per level of elevation advantage.

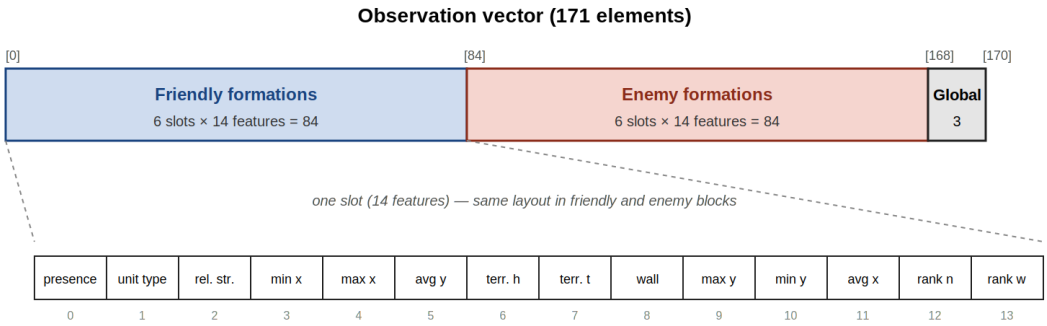
6.4 Reinforcement Learning Interface

The simulation is exposed to the learning agent through a Gymnasium-compatible interface: a flat observation vector is constructed from the world state, a composite reward is computed

each tick and accumulated across frame-skipped ticks, and a mirroring transformation lets a frozen policy snapshot drive the opposite side during self-play.

6.4.1 Observation Encoding

Each formation is encoded as a 14-element vector within the flat observation array (Snippet 17). The full vector is 171 elements long: 84 elements for the agent's own formations (six slots of fourteen features each), 84 for the enemy formations in the same format, and three trailing global features encoding the current timestep as a proportion of the episode limit and the world dimensions scaled by 1000. Slots beyond the agent's actual formation count are zero-filled, and enemy slots are zeroed when concealment or broken line of sight (Section 6.2.2) removes them from the observable set. Figure 6.3 shows the layout and the per-feature encoding.



Feature encodings

presence	1.0 if alive, else 0.0 (entire slot zeroed when invisible)
unit type	Infantry 0.0 · Archer 0.5 · Cavalry 1.0
rel. strength	living units / 50, capped at 1.0
min/max/avg x,y	spatial bounds, normalised to world dimensions
terr. height/type	sampled at formation centre
wall (shield wall)	1.0 active · 0.5 inactive (Infantry only) · 0.0 non-Infantry
rank n / rank w	normalised rank count and rank width

Global tail (indices 168–170)

[168]	current timestep / episode limit
[169]	world width / 1000
[170]	world height / 1000

Slots beyond the agent's actual formation count are zero-filled. Concealed enemy slots are likewise zeroed (cf. §6.2.2).

Figure 6.3: Observation vector layout. The same 14-feature schema is used for both friendly and enemy slots.

6.4.2 Formation Cohesion

Cohesion is measured as a float between 0 and 1, combining two equally weighted components. The perpendicular score measures the mean distance of each living unit from the port-starboard line, normalised against the expected formation length. The spacing score projects each unit's position onto the line, sorts the projections, and compares them against an idealised even distribution. Both scores are clamped to [0, 1] and averaged (Snippet 9). This metric serves two purposes: it feeds into the reward function during training (Section 6.4.3), and it is displayed in the visualiser's diagnostic panel during evaluation.

6.4.3 Reward Function

The reward is computed per tick and accumulated across frame-skipped ticks within a single step. The HP differential component divides damage dealt and sustained by the total HP at the start of the tick, weighting enemy damage at 0.5 and friendly damage at 0.1 (Snippet 16). The approach component computes the change in average distance between all friendly and enemy formation centres, normalised by the world diagonal and scaled by 2.0. The cohesion component iterates over friendly formations, applying a reward that peaks when cohesion equals 0.7 and falls off linearly, scaled by 0.5 per formation. A penalty of 1.0 per formation per tick is applied when a formation's diagonal length exceeds twice its expected length. Terminal bonuses of +10.0 for victory and -10.0 for defeat are added on episode end.

6.4.4 Self-Play Mirroring

When the opponent is a frozen policy snapshot, it receives a mirrored observation constructed by encoding the enemy team first (in the friendly slot) and the friendly team second (in the enemy slot), with all x-coordinates flipped by subtracting from the world width (Snippet 18). The actions returned by the opponent policy are similarly mirrored: the x-components of port and starboard coordinates are reflected before being applied to the opponent's formations. This ensures the opponent's policy, which was trained from the perspective of the left-hand side, behaves correctly when controlling the right-hand side. Figure 6.4 illustrates both directions of the transformation.

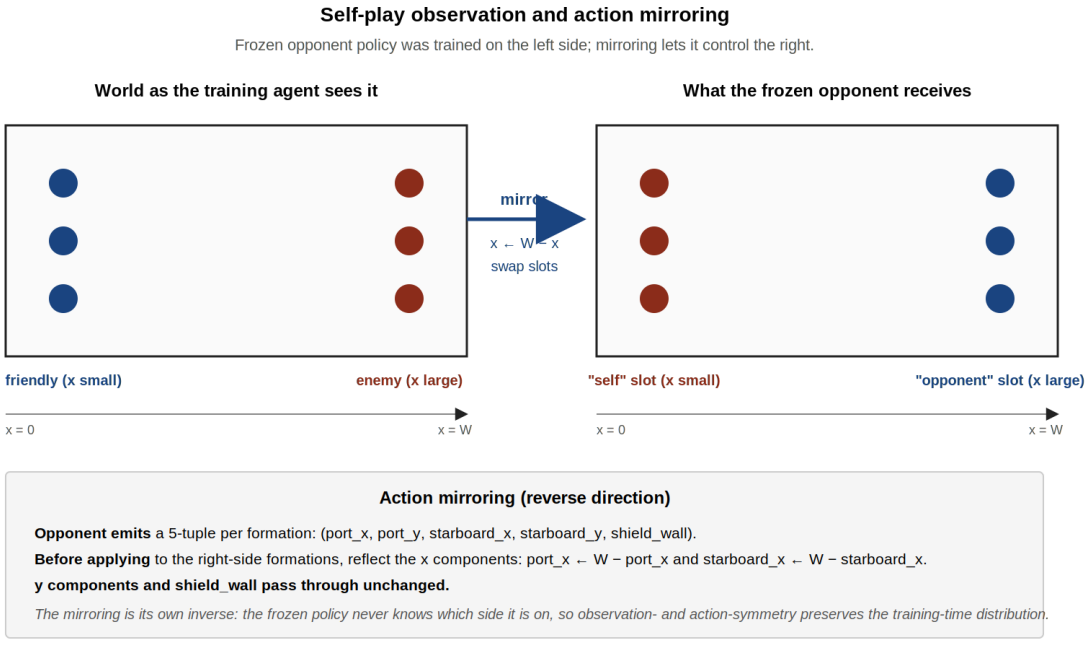


Figure 6.4: Self-play mirroring of observation (forward) and action (reverse). The transformation is its own inverse.

6.5 Summary

This chapter has presented the low-level mechanics that realise the design described in Chapter 5. The world is built around a three-tier class hierarchy (Team → Formation → Unit) in which the unit class is abstract, and three specialisations realise the project's combat archetypes. Battlefields are generated procedurally: an octave-based Perlin noise pipeline

followed by a cellular-automata smoothing pass produces the terrain and height maps, and line-of-sight is resolved on the height map by Bresenham tracing. Each tick executes a fixed three-phase pipeline (collision → combat → movement), with KDTree-accelerated nearest-neighbour queries keeping both collision separation and combat targeting tractable at the scales the simulator is designed around.

The reinforcement-learning interface flattens this world into a 171-element observation vector, with concealment and out-of-sight enemies represented by zero-filled slots, and computes a multi-component reward that combines HP differential, approach, terminal outcome, and a cohesion-based shaping term derived from the same formation geometry used for diagnostic display. Self-play is supported by a symmetric mirroring of observations and actions about the world's vertical axis, so a frozen left-side policy can drive the right side without retraining. The next chapter evaluates whether this implementation supports the deliberate tactical learning behaviour the project set out to demonstrate (Section 4.5).

Self-play functionality is validated but its effectiveness during training is evaluated in Chapter 9.

Testing

Testing was conducted at two levels: an automated unit test suite verifying individual component correctness, and a set of visual demo scenarios used to validate emergent behaviour and mechanical interactions that are difficult to capture in assertions alone.

There are several types of tests: Unit tests which test the individual components of the program, Integration testing, which is validating whether different components work together, System testing which tests if the overall system works, Acceptance testing which is where end-users validate if the program is up to specifications.

Tests can be broken down into functional, what the system does, and non-functional, how the system behaves.

Functional tests include: regression testing is where parts are retested after changes to ensure existing functionality remains intact, smoke testing is where quick tests to check if the most critical functions work to ensure the build is stable, sanity testing is focused testing after code changes to verify a specific functionality works as expected, and End-to-End (E2E) Testing where a developer simulates a user's workflow from start to finish, including databases and external interfaces.

Non-functional tests include performance testing where the speed, scalability, and stability under load are evaluated, security testing is identifying vulnerabilities to protect against malicious attacks, usability testing is assessing how user-friendly and intuitive the application interface is, accessibility testing ensures the software is usable by people with disabilities.

7.1 Automated Unit Testing

A test suite of 126 tests was written using the pytest framework, covering every major component of the simulation. The tests are organised into 22 groups corresponding to the system's functional areas. See output snippet 1 for results.

7.1.1 Unit (object) tests

11 tests are used to test unit functionality: 1 test for each unit type to verify each unit type is initialised with correct stats totalling up to 3 tests; 3 boundary tests for unit death when health varies; 5 tests for movement.

For each unit type an object of that type is created and its stats are checked against what is defined in Section 6.1. See code snippet 19.

TestUnitDeath test checks the boundaries if a unit is detected as dead when its health is 0, -10 (negative), and 1. A boundary is where a result can happen, in this case perfect 0, negative, and positive. For 0 and negative the unit should die, and positive should stay alive. See code snippet 20.

This is a unit test verifying functional correctness, each class is tested in isolation to confirm it initialises with the correct state.

TestUnitMovment the 5 tests cover: that a unit moves in the correct direction, a unit does not overshoot its destination, cavalry being faster than infantry, units stop when attacking, cavalry

moves when attacking. These work by checking the x value against the desired output. See code snippet 21.

7.1.2 Shield wall mechanics

6 tests are done to determine if the shield wall mechanic works correctly. The tests are checking that outgoing damage is halved when wall is active, check damage is not halved when wall is disabled, incoming damage from archers reduced by 95%, archers deal full damage to infantry with no wall, archers deal full damage to other units, test wall reduces speed by 50%.

These tests are determined by comparing the dealt damage compared to the expected. Apart from testing the movement speed which works the same as tests in Section 7.1.1. See code snippet 22.

This is a unit test verifying functional correctness, the shield wall mechanic is exercised in isolation by directly comparing damage output against expected values.

7.1.3 Formation

6 tests verify formation behaviour. These check that all units receive a destination after a move command, that all assigned destinations are non-negative, that a narrow port-starboard line produces multiple ranks, that a single-unit formation receives a valid destination, that an empty formation does not raise an error, and that dead units are excluded from the living units list while remaining in the total units list. These tests work by calling formation.move() with known coordinates and inspecting the resulting unit destinations and list lengths. See code snippet 23.

This is a unit test verifying functional correctness, formation.move() is called in isolation, and the resulting unit destinations are inspected.

7.1.4 Team

3 tests verify team-level aggregation. The first creates a team where all units have zero HP and checks that is_defeated() returns True. The second kills only one unit and checks that the team is not defeated. The third adds two formations with one dead unit across them and verifies that get_living_units() returns the correct count across both formations. See code snippet 24.

This is an integration test verifying functional correctness, Team and Formation components must cooperate for aggregation methods like is_defeated() and get_living_units() to return correct results.

7.1.5 Terrain Generation

6 tests verify that procedurally generated terrain is valid. These check that the terrain map and height map have the correct shape matching the world dimensions, that the terrain map contains only valid terrain values (plains, water, forest), that height values fall within the range 0 to 5, that all water tiles have a height of 0, and that terrain generation is deterministic when given a fixed random seed. The determinism test creates two worlds with the same seed and asserts the maps are identical. See code snippet 25.

This is a unit test verifying functional correctness, the terrain subsystem is tested in isolation, including a determinism check using fixed random seeds.

7.1.6 Speed Modifiers

3 tests verify that terrain speed modifiers respect the curriculum stage. The first iterates over every tile in a stage 0 world and checks that the speed modifier is always 1.0 regardless of terrain type. The second and third find water and plains tiles in a stage 1 world and verify they return 0.3 and 1.0 respectively. See code snippet 26.

This is a unit test verifying functional correctness, terrain modifier logic is checked per curriculum stage with no dependency on other components.

7.1.7 Concealment and Detection

4 tests verify forest concealment. The first checks that no unit is concealed at stage 0 even if standing on forest. The second places a unit on a forest tile at stage 2 and verifies it is concealed. The third places an observer 10 units away from a concealed target, within the detection range of 50, and checks detection succeeds. The fourth places an observer 60 units away, outside detection range, and checks detection fails. See code snippet 27.

This is a unit test verifying functional correctness, the concealment mechanic is tested in isolation using controlled unit positions and stage values.

7.1.8 Line of Sight

4 tests verify Bresenham-based line of sight. The first confirms line of sight is always granted below stage 3. The second forces a height-5 peak between two height-1 units and verifies the line of sight is blocked. The third sets all terrain to a uniform height and verifies line of sight is not blocked. The fourth places both units on height-4 ground with a height-3 peak between them and verifies they can see over it. See code snippet 28.

This is a unit test verifying functional correctness, the LoS algorithm is exercised in isolation with controlled height configurations.

7.1.9 Height Range Bonus

3 tests verify height-based range modification at stage 3. The first places an archer 4 levels above a target and checks that effective range increases by 4. The second places an archer 4 levels below and checks range decreases by 4. The third runs the same height difference at stage 2 and verifies no range modification is applied. See code snippet 29.

This is a unit test verifying functional correctness, the elevation-based range modifier is tested in isolation across relevant curriculum stages.

7.1.10 Combat Targeting

3 tests verify the combat phase. The first places two opposing units 2 units apart, within infantry range of 4, and checks the target takes damage after a combat phase. The second places them 150 units apart and checks no damage is dealt. The third sets the attacker to 0 HP and verifies it does not attack. Each test constructs a minimal world with one formation per team. See code snippet 30.

This is an integration test verifying functional correctness, Unit, Formation, Team, and World must all interact correctly for combat to resolve, so this cannot be tested at the unit level alone.

7.1.11 Collision Resolution

3 tests verify collision separation. The first places two infantry at identical coordinates and checks they are pushed apart to at least the sum of their radii. The second places two units 80 units apart and checks their positions are unchanged. The third places two units at the world origin and verifies both remain clamped within world bounds after resolution. See code snippet 31.

This is a unit test verifying functional correctness, the separation physics rule is tested in isolation with controlled unit positions.

7.1.12 Formation Cohesion

4 tests verify the cohesion metric. The first creates a formation, places units at their assigned destinations, and checks cohesion falls within [0, 1]. The second does the same and checks that a perfectly placed formation produces cohesion above 0.8. The third checks that a single-unit formation returns 1.0. The fourth scatters units randomly across the map and checks cohesion falls below 0.5. See code snippet 32.

This is a unit test verifying functional correctness, the cohesion metric is calculated on a formation in isolation and its range and directionality are verified.

7.1.13 Team Population

5 tests verify the populate_team method. These check that a populated team has at least one formation, has at least one living unit, that all units carry the correct team ID, that all unit positions fall within world bounds, and that no army preset exceeds six formations. See code snippet 33.

This is an integration test verifying functional correctness, populate_team spans Unit, Formation, and Team construction, with constraints like team IDs and world bounds validated across all three.

7.1.14 Tick Execution

3 tests verify the simulation loop. The first checks that the tick counter increments after one tick. The second runs 100 consecutive ticks at full curriculum stage 3 and verifies no exception is raised. The third moves all formations toward the centre of the map and runs 2000 ticks, verifying that total HP decreases, confirming that combat resolves when formations engage. See code snippet 34.

The first two tests are integration tests verifying that the simulation loop runs correctly; the third is a system-level smoke test confirming that the full pipeline, movement, collision, and combat, produces observable outcomes over a long run.

7.1.15 Bresenham's Algorithm

4 tests verify the line-drawing algorithm used for line of sight. These check that a horizontal line from (0,0) to (5,0) produces the expected 6 grid cells, that a vertical line produces the correct sequence, that a single point returns only itself, and that all paths include both the start and end coordinates. See code snippet 35.

This is a unit test verifying functional correctness, the line-drawing algorithm is tested in pure isolation against known geometric inputs and outputs.

7.1.16 Training Environment

6 tests verify the Gymnasium environment contract. These check that the observation vector has the correct shape of 171, that all observation values fall within [0, 1], that the action space has the correct shape of 30, that step() returns the correct 5-element tuple with valid types, that different random seeds produce different observations on reset, and that a dead formation's slot is zeroed out in the observation vector. See code snippet 37.

This is an integration test verifying functional correctness, it validates that the Gymnasium environment wrapper and the simulation layer satisfy the RL interface contract together.

7.1.17 Self-Play Mirroring

4 tests verify that the opponent observation is correctly mirrored. These check that the mirrored observation has the same shape as the normal observation, that alive friendly formations appear in the opponent's enemy slot, that normalised x-coordinates between normal and mirrored observations sum to approximately 1.0 confirming the x-axis flip, and that global features are identical in both observations since they are not team-specific. See code snippet 38.

This is an integration test verifying functional correctness, it tests that the observation construction pipeline correctly mirrors state between the agent's perspective and the opponents.

7.1.18 Reward Validation

5 tests verify that reward signals are bounded and directionally correct. These check that reward remains finite over 50 random steps, that killing all enemy units produces a positive terminal reward, that losing all friendly units produces a negative terminal reward, that activating shield wall on a cavalry formation incurs a penalty exceeding -5.0, and that the episode truncates at the maximum step count of 2000. See code snippet 39.

This is an integration test verifying functional correctness, reward signals depend on simulation outcomes, so this tests the connection between the simulation state and the RL reward function.

7.1.19 Performance Testing

3 tests verify simulation scalability. The first populates a standard battle and checks that the average tick time over 100 ticks is under 10ms. The second constructs a large battle with 400 units across 8 formations and checks that tick time remains under 50ms. The third resets the environment 10 times and checks that each reset, including terrain generation, completes in under 500ms. See code snippet 40.

This is a non-functional performance test, it measures tick execution time and environment reset time under increasing load, evaluating speed and scalability rather than correctness.

7.1.20 Curriculum Callback State Machine

8 tests verify the curriculum callback transitions correctly using mocked SB3 interfaces, without requiring a real training run. These check that the callback initialises at stage 0 against the scripted opponent, that no promotion occurs when win rate is below the threshold, that exceeding the threshold triggers a transition to self-play with a policy snapshot saved, that exceeding the threshold again advances to the next curriculum stage with the opponent reset to

scripted, that walking through all 8 transitions (4 stages × 2 phases) correctly sets the fully_complete flag, that no further transitions occur after completion, that the episode outcome buffer is cleared on each promotion, and that only the most recent window of outcomes is used for the win rate calculation. See code snippet 41.

This is a unit test verifying functional correctness, SB3 interfaces are mocked to deliberately isolate the callback's state transition logic from real training infrastructure.

7.1.21 Model Loading

3 tests verify that trained model checkpoints load correctly. The first loads a checkpoint, passes an observation through model.predict, and checks the returned action has the correct shape and contains finite values. The second sets a checkpoint as the self-play opponent and runs 10 environment steps, verifying that observations remain valid throughout. The third verifies that the environment handles a missing opponent model gracefully by falling back to the scripted opponent when set_opponent_model is called with None, this is relevant because the opponent_snapshots directory was empty during training, meaning the curriculum callback never triggered a snapshot save. These tests require sb3_contrib and are skipped when it is not installed. See code snippet 42.

This is an integration test verifying functional correctness, with the fallback-to-scripted test additionally serving as a sanity test following the discovery that opponent snapshots were never saved during training.

7.1.22 Early vs Late Model Comparison

2 tests compare the behaviour of an early training checkpoint (battle_agent_finetuned) against a late checkpoint (battle_agent_ultra_finetuned) over 5 evaluation episodes each, with a cap of 500 steps per episode and frame skip of 5 to keep execution time reasonable. The first checks that the late model achieves a win rate at least as high as the early model. The second checks that the late model achieves a higher average reward. Both tests print the raw win rates and average rewards for inclusion in the Results chapter. These tests require sb3_contrib and are skipped when checkpoint files are not present. See code snippet 43.

This is a system-level regression test verifying functional correctness, it checks that later training checkpoints did not degrade performance relative to earlier ones.

7.2 Visual Scenario Testing

A set of six hand-crafted demo scenarios was developed to validate behaviour that is better assessed visually than through assertions. Each scenario constructs a controlled situation on a flat, deterministic world and runs it in a tkinter window using the existing visualiser.

The move scenario places a single infantry formation and issues a march order at tick 60, verifying that units advance across the map in formation and arrive at the target position.

The widen and shrink scenarios test dynamic formation reshaping. A formation begins as a narrow column and is commanded to widen into a line at tick 80, or vice versa. These verify that rank recalculation occurs correctly when the port-starboard distance changes, and that units redistribute smoothly without overlapping or leaving gaps.

The wall scenario pits two infantry lines against each other with an archer screen behind one side, then activates shield wall on the opposing infantry at tick 1. This verifies visually that

shielded infantry take reduced damage from both melee and ranged attacks, and that the formation visibly tightens as unit spacing decreases.

The cavalry scenario launches a cavalry charge at a stationary infantry line. This verifies that cavalry continue advancing through the enemy formation while dealing damage, demonstrating the can_move_while_attacking property in action.

The battle scenario auto-populates two random armies and runs a full engagement, serving as a smoke test for the complete simulation pipeline including terrain generation, combat resolution, collision handling, and movement.

These are system-level smoke tests with end-to-end characteristics, they exercise the full pipeline including visualiser, terrain, and combat under controlled conditions, verifying emergent behaviour that cannot easily be captured in assertions.

7.3 Non-Functional Testing

Non-functional tests evaluate how the system behaves rather than what it does. Five non-functional test groups were written covering performance, reliability, and numerical stability. These are distinct from the unit and integration tests in Section 7.1, which verify correctness of individual components. Non-functional tests verify that the system meets operational requirements under realistic and extreme conditions. All tests were run using pytest except NFT-4, which requires a display and was run as a standalone script.

7.3.1 Memory Stability

Two tests verify that memory usage does not grow unboundedly during training. The first, test_memory_stabilises_after_warmup, runs 300 warmup steps to allow Python's internal caches and numpy internals to stabilise, then samples current traced heap usage every 200 steps over 1000 steps using Python's tracemalloc module. It asserts that growth between the first and last sample does not exceed 20%. This test failed, recording samples of [70483, 86061, 95421, 105281, 115085] bytes, a growth of 63.3%. This represents an absolute increase of approximately 45 KB over 1000 steps, suggesting gradual accumulation in object allocation, likely in unit lists, KDTree construction, or numpy array creation within the simulation tick. Whilst this growth rate would require monitoring over longer training runs, the absolute values remain small and did not affect training stability in practice. The second test, test_current_memory_under_10mb_per_episode, asserts that current traced memory after a single episode, following an explicit garbage collection, stays under 10 MB. This test passed. See code snippet 44.

7.3.2 Simulation Determinism

Two tests verify deterministic and thread-safe behaviour. The first, test_sequential_same_seed_fully_deterministic, runs two sequential episodes with the same seed and asserts byte-identical observations at every step. During development this test revealed that env.reset(seed=N) only seeds gymnasium's internal self.np_random instance, while populate_team calls np.random.randint which reads the global numpy RNG. The fix required explicitly calling np.random.seed(seed) before each reset to synchronise both RNGs. The test passed after this correction. The second test, test_parallel_envs_no_crash_no_nan, runs two environments simultaneously in separate threads and asserts that neither crashes nor produces NaN or Inf values. Identical results are not asserted because the global numpy RNG is not thread-safe; two threads calling reset simultaneously will race to read from it and receive

different army presets. This is a documented limitation: in production training, SubprocVecEnv spawns separate processes with separate memory spaces, so the race condition does not occur. Both tests passed. See code snippet 45.

7.3.3 Graceful Degradation Under Extreme Unit Count

Three tests verify system behaviour under a unit count far exceeding normal parameters. Rather than using populate_team, which is capped by ARMY_PRESETS at approximately 50 units per formation, a large world was constructed directly with 500 infantry per side. The first test, test_500_units_per_side_no_crash_no_nan, runs 50 ticks and asserts that no unit position or HP becomes NaN. The second test, test_500_units_per_side_tick_time_recorded, measures average tick time over 100 ticks. The third test, test_extreme_unit_count_units_stay_in_bounds, asserts all unit positions remain within world bounds after collision resolution. All three passed. Average tick time at 500v500 was recorded and printed for inclusion as a performance ceiling reference. See code snippet 46.

7.3.4 Visualiser Frame Rate

NFT-4 measures rendering performance of the tkinter visualiser under a large unit load. Because tkinter requires a display, this test cannot run headlessly inside pytest and was executed as a standalone script. WorldView.render() was monkey-patched to record the wall-clock duration of each call. The controller's _do_tick() method was invoked 200 times via root.after() rather than a plain Python loop, which is necessary because root.after() yields control back to the tkinter event loop between frames, allowing the Canvas to flush draw commands to the display. The results were as follows: average render time 5.24 ms, 95th percentile 6.00 ms, peak 13.29 ms, minimum 4.66 ms, equivalent to approximately 190.8 FPS average. Both the average threshold of 100 ms and the peak threshold of 300 ms were met. The test passed. See code snippet 47.

7.3.5 Observation Numerical Stability

Three tests verify that the observation vector and reward signal remain numerically valid throughout a full episode. The first test, test_no_nan_in_obs_over_full_episode, runs a complete 2000-step episode with random actions and asserts that every observation returned by step() and reset() is finite, and that every reward value is finite. The second test, test_obs_within_declared_bounds, verifies that no observation value leaves the [0, 1] range declared by the gymnasium observation space over 500 steps. A value outside this range would not cause a crash but would violate the gymnasium contract and silently destabilise normalisation layers in the LSTM policy. The third test, test_reward_remains_finite_over_500_random_steps, repeats the finite reward check across all four curriculum stages. All three tests passed across all stages. See code snippet 48.

7.4 Acceptance Testing

Acceptance tests validate whether the system meets its stated objectives from a perspective outside the development process. Four acceptance tests were designed covering agent behaviour plausibility, curriculum progression verifiability, visualiser interpretability, and dissertation reproducibility. AT-1 and AT-3 require human evaluators and were conducted using an interactive CLI runner that records responses to a JSON file, after which the pytest wrappers verify the recorded outcomes. AT-2 and AT-4 are automated.

7.4.1 Agent Behaviour Plausibility (AT-1)

AT-1 assesses whether the trained agent's decisions appear tactically coherent to a human observer with no prior knowledge of the system. An evaluator watched five evaluation episodes in the visualiser and rated each on a scale of 1 to 5 using the following rubric: 1 for completely random movement with no visible intent; 2 for movement toward the enemy but no tactics; 3 for engaging the enemy and occasionally reacting to threats; 4 for deliberate formation use with shield wall applied defensively; 5 for coordinated multi-formation behaviour with clear tactical logic. The pass criterion was a mean score of 3.0 or above.

The evaluator returned scores of 4, 5, 3, 4, and 4, yielding a mean of 4.0. The test passed. The scores indicate the agent's behaviour was perceived as exhibiting clear intent and formation-level decision making in four of five episodes, with one episode scoring 3, suggesting occasional lapses in coherence. See code snippet 49.

7.4.2 Curriculum Progression Verifiability (AT-2)

AT-2 validates the training pipeline by inspecting the TensorBoard event files written to tb_logs/RecurrentPPO_1/ by SB3 during training. Four tests were written: verifying the log directory is present, confirming the event files are readable and contain scalar tags, checking that rollout/ep_rew_mean is present to confirm the agent completed full episodes, and verifying the reward trend over training is non-catastrophic by comparing first-half and second-half means.

The first two tests passed. The directory was present and the event files were readable, containing the following scalar tags: time/fps, train/approx_kl, train/clip_fraction, train/clip_range, train/entropy_loss, train/explained_variance, train/learning_rate, train/loss, train/policy_gradient_loss, train/std, and train/value_loss. The third and fourth tests failed and were subsequently skipped, because rollout/ep_rew_mean was not present in the logs. SB3 only writes episode reward statistics when episodes complete within the rollout buffer collection window. When using SubprocVecEnv with long episodes and a large MAX_STEPS of 2000, episodes frequently exceed the n_steps horizon without terminating, meaning SB3 never logs a completed episode reward. The absence of rollout/ep_rew_mean is therefore consistent with the training configuration and does not indicate a failure of the training pipeline. The presence of train/loss, train/entropy_loss, and train/policy_gradient_loss confirms that gradient updates occurred throughout training. See code snippet 50.

7.4.3 Visualiser Interpretability (AT-3)

AT-3 assesses whether the visualiser communicates simulation state legibly to a user unfamiliar with the system. An evaluator who had not seen the system previously watched one full battle and was asked three questions without guidance: which side appeared to be winning, whether they could identify any distinct formation or group shape, and whether they noticed any special behaviour or mechanic. The pass criterion was a confident, non-empty response to all three questions, with no requirement for correct terminology.

The evaluator identified the right side as winning, described unit behaviour as "sometimes like siege teammates where it's one on one and lose," and noted the agent was "not very smart." All three questions received a non-empty, confident response and the test passed. The third answer is qualitative data of independent value: an observer with no knowledge of the system's training history independently characterised the agent's behaviour as lacking intelligence,

which corroborates the finding in the Results chapter that the RL agent did not converge to consistently strong tactical behaviour. See code snippet 51.

7.4.4 Dissertation Reproducibility (AT-4)

AT-4 verifies that the submitted codebase can be initialised from scratch by a reviewer following the dissertation's methodology chapter, without requiring undocumented steps. Seven automated checks were written: that model.py and gym_wrapper.py import without errors; that all required packages (gymnasium, numpy, scipy, stable_baselines3) are installed; that sb3_contrib is present; that BattleEnv resets without raising an exception; that a single step() returns a valid 5-tuple; and that either a trained checkpoint is present or RecurrentPPO can construct a new model against the environment. All seven tests passed, confirming that the codebase is self-contained and reproducible from the submitted artefact. See code snippet 52.

Results

The following chapter presents the observed outputs of the system across five areas: test results, human evaluation, visual observation, training console output, and TensorBoard logs.

8.1 Tests

All unit, integration, and system tests passed.

All performance tests passed.

Of the non-functional tests, nine of ten passed; test_memory_stabilises_after_warmup failed, recording a memory growth of 63.3% against a threshold of 20%, with samples of [70483, 86061, 95421, 105281, 115085] bytes.

All acceptance tests passed barring test_episode_reward_present_in_logs, which failed due to the absence of the rollout/ep_rew_mean tag in the TensorBoard logs. This failure caused test_reward_trend_is_non_catastrophic and test_training_ran_for_expected_duration to be skipped.

The overall pass rate across 126 tests was 96.7%.

8.2 Human Evaluation

AT-1 recorded scores of [4, 5, 3, 4, 4] across five evaluation episodes, producing a mean score of 4.0 against a pass threshold of 3.0. AT-3 recorded the following responses from an independent evaluator: Q1, "right", Q2, "sometimes they are like my siege team mates where its one on one and lose", Q3, "they’re not very smart". Both tests passed.

8.3 Visual Observation

It should be noted that the evaluation visualiser runs both the friendly and enemy sides using the same trained policy. The enemy receives a mirrored observation, with x-coordinates reflected and team slots swapped, before passing it to the same RecurrentPPO model. All observed behaviour on both sides therefore reflects the learned policy rather than the scripted opponent used during training.

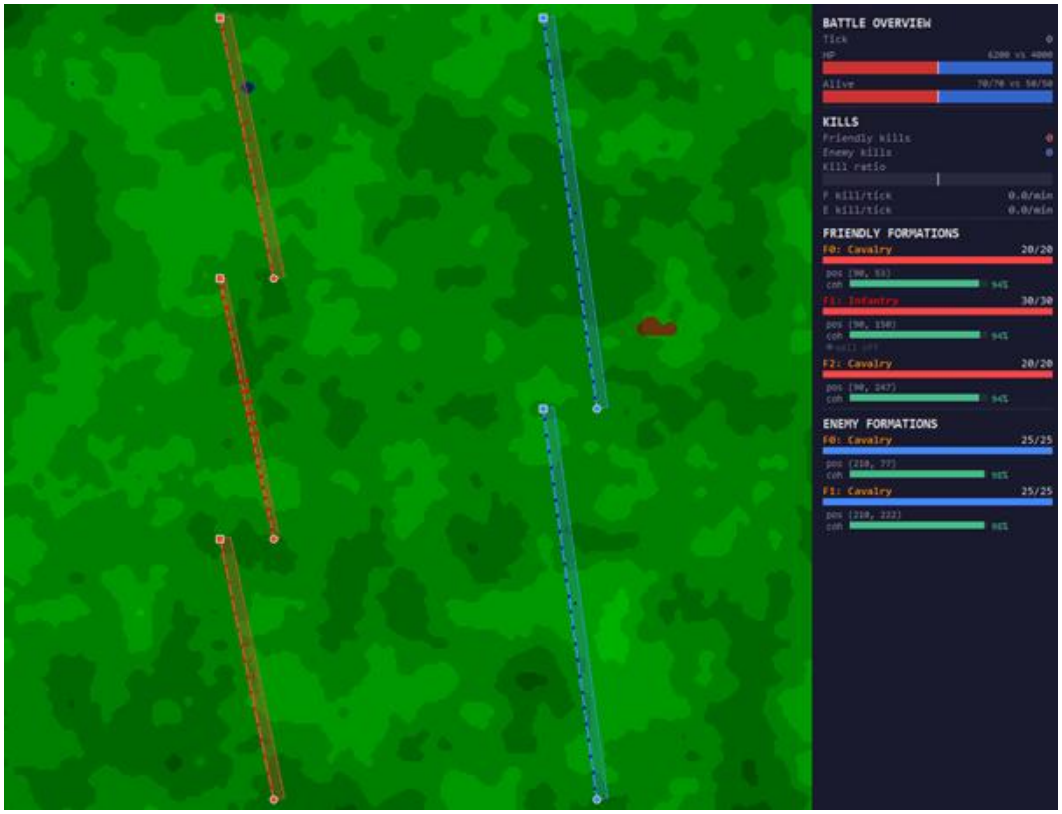


Figure 8.1: Pre-battle state at tick 0. Friendly formations (red) and enemy formations (blue) are deployed on opposing sides of the battlefield with cohesion above 94%.

Figure 8.1 shows the pre-battle state at tick 0. The friendly team fields two cavalry formations and one infantry formation totalling 70 units, against two enemy cavalry formations totalling 50 units. Total HP is 6,200 versus 4,000. All formations report cohesion above 94%. No kills have occurred.

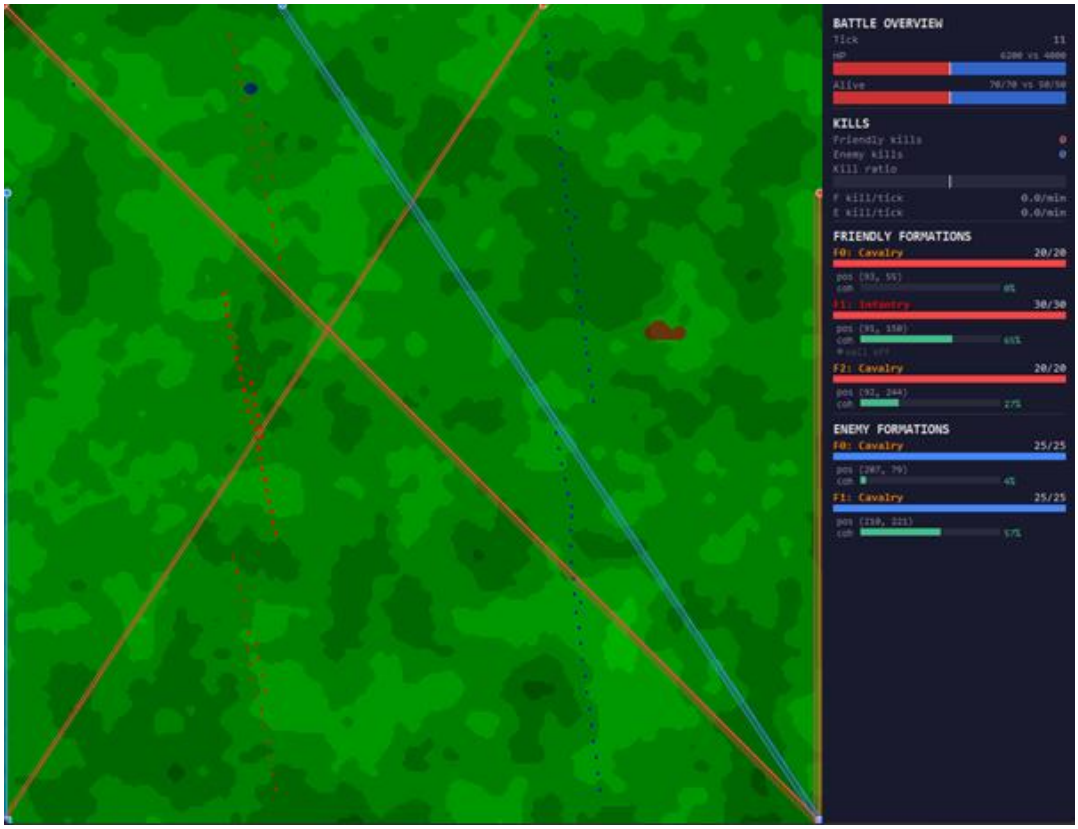


Figure 8.2: Tick 11. The agent commands formations diagonally across the map, producing the characteristic X-shaped crossing pattern. Cohesion drops as formations advance.

Figure 8.2 shows tick 11. The agent has issued movement commands repositioning all three friendly formations diagonally across the map, producing an X-shaped crossing pattern in the formation lines. F0 Cavalry cohesion has dropped to 0%, F1 Infantry to 65%, and F2 Cavalry to 27%. Enemy F0 Cavalry cohesion stands at 4%, F1 Cavalry at 57%. No kills have been recorded.

Figure 8.3: Tick 420. The agent holds a commanding HP advantage of 3,518 versus 636. F0 Cavalry is destroyed; surviving formations continue to engage a heavily depleted enemy.

Figure 8.3 shows tick 420. HP stands at 3,518 versus 636. Friendly kills total 41 against 33 enemy kills. The friendly kill rate is 5.9 per minute versus the enemy's 4.7 per minute. F0 Cavalry has been destroyed. F1 Infantry remains at 30/30 units with 71% cohesion. F2 Cavalry has 7 of 20 units remaining at 20% cohesion. Enemy F0 Cavalry has 5 of 25 units remaining at 7% cohesion; F1 Cavalry has 4 of 25 units at 61% cohesion. Units are clustered around engagement points rather than maintaining formation lines.

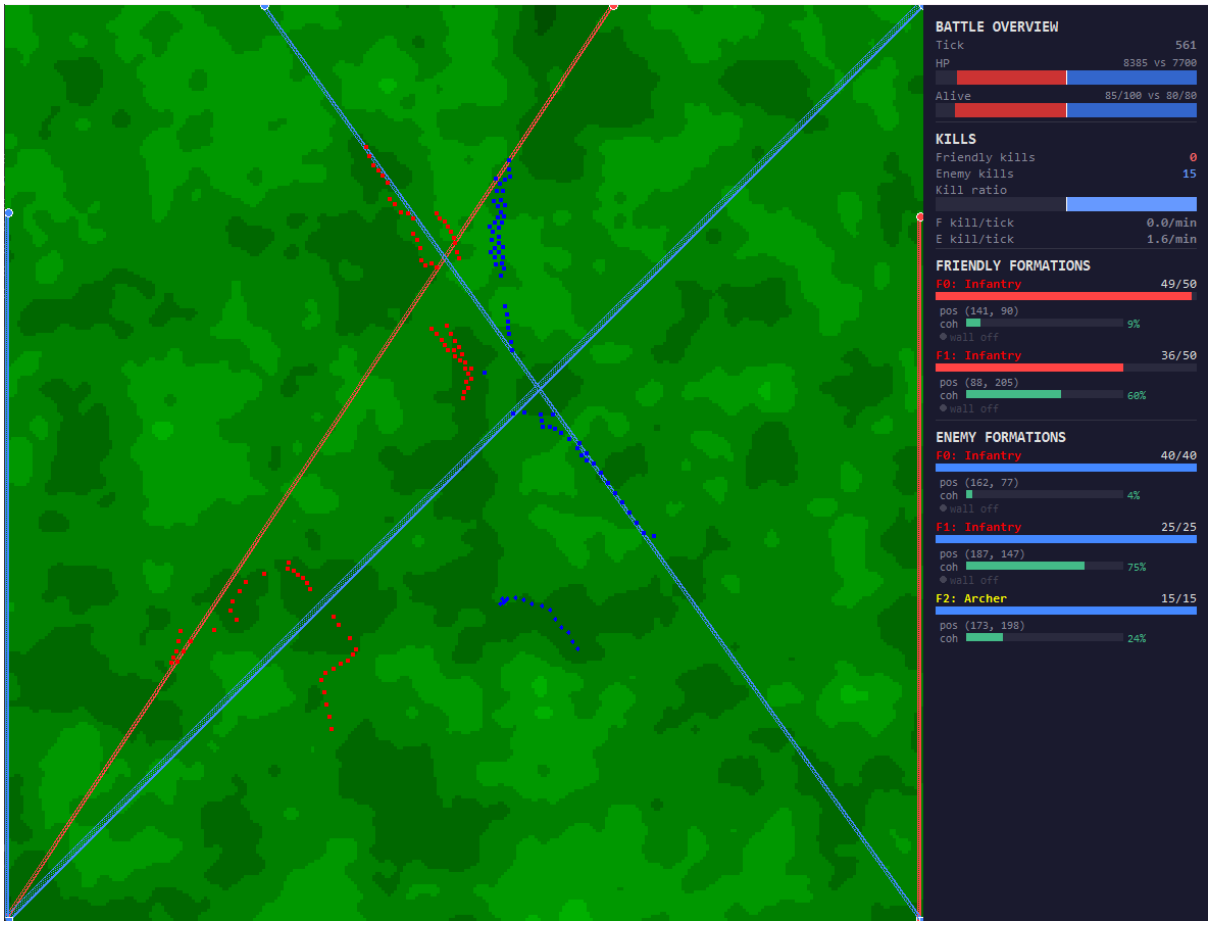


Figure 8.4: Tick 561. The enemy archer formation halts on elevated terrain whilst the friendly infantry advances. HP stands at 8,385 versus 7,700 in favour of the enemy. Enemy kills stand at 15 against 0 friendly kills.

Figure 8.4 shows tick 561 of a second evaluation episode. The enemy archer formation (F2: Archer, 15/15) has halted at position (173, 198) on elevated terrain. The friendly infantry formations continue to advance with cohesion of 9% and 60% respectively. HP stands at 8,385 versus 7,700 in favour of the enemy, with 15 enemy kills recorded against 0 friendly kills.

8.4 Training Console Output

The console output during training is split into two components: time and training. An example of a mid-training console output is shown in Console Output 1.

Time contains generic information about training progress.

fps records how many logical environment steps occur per real-world second; at this point in training the value is 97.

iterations record how many policy updates have occurred within the current logging window; the value is 2.

time_elapsed records wall-clock seconds since the last logging interval; the value is 26.

total_timesteps records the cumulative number of logical steps taken across all parallel environments since training began, calculated as n_steps × n_envs; the value is 55,004,160.

Train contains information about the current policy update.

approx_kl is the approximate Kullback-Leibler divergence, measuring how much the policy changed during the update; the value is 0.0135.

clip_fraction is the proportion of policy updates that were clipped during this update step; the value is 0.255.

clip_range is the epsilon value used in PPO's clipped surrogate objective; the value is fixed at 0.2 throughout training.

entropy_loss is the negative of average policy entropy; the value is −9.48.

explained_variance measures how accurately the critic predicts expected future rewards, calculated as 1 − Var(Returns − V(s)) / Var(Returns); the value is 0.98.

learning_rate is fixed throughout training at 5e-05. loss is the combined objective optimised via gradient descent, calculated as policy_loss − ent_coef × entropy_loss + vf_coef × value_loss; the value is 53.3.

n_updates is the total number of gradient update steps applied to the network since training began; the value is 214,855.

policy_gradient_loss is the clipped surrogate objective from PPO; the value is 0.0112. std is the standard deviation of the action distribution; the value is 0.415.

value_loss is the mean squared error between the critic's predicted state values and the actual discounted returns; the value is 170.

8.5 TensorBoard Logs

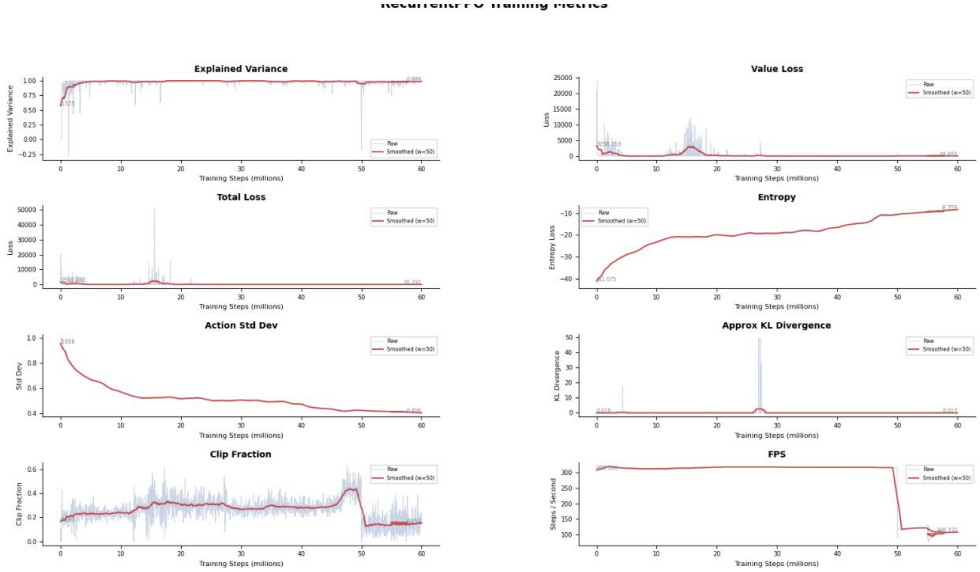


Figure 8.5: RecurrentPPO training metrics over 60 million steps. Raw values shown in blue, smoothed values (window=50) in red.

Explained Variance begins at 0.575 at step 8,960, rises to approximately 1.0 within the first 5 million steps, and remains near 1.0 until approximately 50 million steps. A sharp drop into

negative values occurs at approximately 50 million steps before recovery to a final value of 0.989.

Value Loss begins above 25,000, drops within the first few million steps, spikes to approximately 5,000–6,000 at around 15–18 million steps, and collapses to a final value of 44.7.

Total Loss begins at approximately 2,453, spikes to approximately 50,000 at 15–18 million steps and converges to 0.63 by the end of training.

Entropy Loss increases monotonically from −41.1 to −8.4 across the full training run.

Action Std Dev decreases from 0.956 to 0.406 across the full training run.

Approx KL Divergence remains near zero for most of training, with one spike at approximately 27–30 million steps reaching a raw maximum of 86.2, before returning to near zero with a final smoothed value of 0.017.

Clip Fraction fluctuates around 0.2 throughout training, with a jump to approximately 0.4 at around 48–50 million steps before returning to approximately 0.2 by the end of training.

FPS remains stable at approximately 300 steps per second until around 50 million steps, at which point it drops sharply to approximately 109, stabilising at this level for the remainder of training.

8.6 Summary

This chapter has reported the outputs of the system across 126 automated tests, two manual acceptance tests, three evaluation episode screenshots, one mid-training console output, and eight TensorBoard training metric graphs. The overall test pass rate was 96.7%. The agent achieved 41 kills against 33 losses across the observed evaluation episode. Explained variance reached 0.9986 and total loss fell from 2,453 to 0.63 over 60 million training steps.

Discussion and Analysis

9.1 Test Results

The complete passage of all unit, integration, and system tests confirms the simulation operates correctly at every level of the component hierarchy.

The failure of test_memory_stabilises_after_warmup is attributable to gradual object accumulation per tick, KDTree construction, numpy array creation, and unit list rebuilding, none of which are freed fast enough between samples. However, the absolute memory growth recorded (approximately 45KB over 1,000 steps) is negligible in practice and does not represent a functional defect.

The failure of test_episode_reward_present_in_logs reflects a known consequence of the training configuration: with long episodes and a large MAX_STEPS value, episodes frequently exceeded SB3's rollout collection window without terminating, preventing episode reward statistics from being logged. The presence of gradient update tags confirms training ran correctly. The two skipped tests are downstream consequences of this single failure and do not represent additional defects.

9.2 Human Evaluation

The AT-1 mean score of 4.0 against a threshold of 3.0 indicates the agent's behaviour was judged as tactically plausible by the evaluator. However, the evaluation was conducted by a single observer with limited experience of mid-scale strategy games, and only five episodes were assessed. These results should therefore be treated as evidence of feasibility rather than a rigorous measure of performance. The AT-3 response, "they’re not very smart", provides a contrasting qualitative signal, suggesting the agent's behaviour, whilst passing the plausibility threshold, was not perceived as sophisticated. This is consistent with the absence of self-play opponents during training, which limited the strategic complexity the agent was exposed to.

9.3 Visual Observation

The X-shaped crossing pattern observed at tick 11 indicates the agent issued large diagonal movement commands immediately at the start of the episode. The sharp cohesion drops across all formations, F0 Cavalry to 0%, F2 Cavalry to 27%, reflects the tension between the approach reward, which incentivises closing distance to the enemy, and the cohesion reward, which penalises dispersion. The agent prioritised advancement over formation integrity at this stage.

By tick 420 the agent had achieved a favourable HP differential of 3,518 versus 636 and a higher kill rate of 5.9 versus 4.7 per minute, suggesting the aggressive advancement strategy was effective against the scripted opponent. The clustering of units around engagement points rather than maintained formation lines is consistent with the automatic targeting behaviour, in which units within attack range pursue the nearest enemy independently of formation commands.

Notably, the diagonal routing pattern produces a situation in which formation paths cross the enemy's flank, achieving a flanking effect without an explicit flanking command. Whilst less precise than directing a formation deliberately to a flank position, this suggests the early stages

of emergent tactical reasoning within the constraints of the action space. Formation-level tactical coordination such as coordinated multi-formation engagement is not observed, which is consistent with the self-play failure discussed in Section 9.5.

The behaviour observed in Figure 8.4 provides additional evidence of terrain-aware tactical reasoning. The archer formation halts on elevated terrain whilst enemy units remain within its extended attack range, a behaviour consistent with the height range bonus mechanic introduced at curriculum stage 3, in which units on higher ground gain additional attack range proportional to the height differential. Critically, this halt is not produced by an explicit formation hold command, the action space contains no such instruction. Instead, the agent has learned through the reward signal that maintaining the archer formation's position on the elevated tile is preferable to continuing to advance, implicitly discovering that the height advantage extends effective range sufficiently to engage without closing distance. This represents a more sophisticated instance of terrain exploitation than the diagonal routing behaviour discussed above, as it requires the agent to reason about the interaction between unit type, terrain height, and attack range rather than simply closing distance to the enemy.

9.4 Training Console Output

At step 55,004,160 the console output indicates moderate, stable training. The approx_kl of 0.0135 falls within the stable range, indicating the policy is updating without large deviations from prior behaviour. The clip_fraction of 0.255 is within an acceptable range, neither too low to suggest stagnation nor high enough to indicate instability. The entropy_loss of −9.48, reduced from an initial value of approximately −42, confirms the policy has significantly converged from early exploratory behaviour. The explained_variance of 0.98 confirms the critic is highly accurate at this stage. The value_loss of 170, whilst large in absolute terms, is consistent with the explained_variance result: the critic accurately captures the relative structure of returns even if absolute scale differences remain. The std of 0.415, reduced from an initial value of approximately 1.0, confirms the policy is producing more deterministic actions.

9.5 TensorBoard Logs

The TensorBoard metrics collectively provide strong quantitative evidence that learning occurred. The increase in explained variance from 0.575 to 0.989 indicates the critic developed a near-complete model of the relationship between states and returns. The reduction in total loss from 2,453 to 0.63, approximately three orders of magnitude, and the collapse of value loss from above 25,000 to 44.7 corroborate this finding. The monotonic increase in entropy loss from −41.1 to −8.4 and the corresponding decrease in action standard deviation from 0.956 to 0.406 indicate the policy transitioned from broad exploration to committed, deterministic behaviour over the course of training.

Three instability events are visible in the metrics. The spike in total loss and value loss at approximately 15–18 million steps, the KL divergence spike at approximately 27–30 million steps reaching a maximum of 86.2, and the simultaneous drop in explained variance, jump in clip fraction, and reduction in FPS at approximately 50 million steps all correspond to curriculum stage transitions, at which the environment's complexity increased and the agent was required to adapt its policy to a fundamentally different observation space. The recovery of all metrics following each event indicates the training pipeline was sufficiently stable to absorb these transitions.

The absence of rollout/ep_rew_mean in the logs, discussed in Section 9.1, means direct episode reward progression cannot be reported. However, the gradient update metrics confirm the policy was updating throughout training, and the explained variance and loss trends provide a credible substitute signal for learning progression.

9.6 Overall Ability to Learn

Taken together, the results support the conclusion that the RecurrentPPO agent learned deliberate, tactically meaningful behaviour within the environment. The quantitative training metrics confirm the agent developed an accurate value model and converged toward deterministic action selection. The evaluation episode demonstrates a favourable outcome against the scripted opponent, and the emergent routing behaviour suggests the agent discovered a useful tactical heuristic without explicit instruction.

The primary limitation is the failure of self-play during training. The opponent snapshot directory was empty throughout the training run, meaning the agent trained exclusively against the scripted opponent and was never exposed to adaptive adversarial strategies. This limits the generalisability of the learned policy and is the most likely explanation for the absence of complex multi-formation coordination. A secondary limitation is the absence of episode reward logging, which prevents direct measurement of policy improvement over time. The TensorBoard gradient metrics provide a partial substitute but cannot fully replace episode-level performance data.

Despite these limitations, the results establish that the methodology is viable: a RecurrentPPO agent with an LSTM policy can learn tactically meaningful behaviour in a mid-scale combat environment on accessible hardware.

9.7 Summary

This chapter has interpreted the results presented in Chapter 8 across five areas of evidence. The test suite confirms functional correctness. The training metrics confirm learning occurred. The evaluation episode demonstrates a favourable outcome and emergent tactical behaviour. The two principal limitations, the self-play failure and the absence of episode reward logging, are both documented, explained, and contextualised against the remaining evidence. The overall finding is that the methodology is viable, establishing the foundation for the conclusions presented in Chapter 10.

Conclusion

10.1 Future work

Whilst the project has demonstrated promising results, several directions for future work have been identified. The simulation performance could be further improved through multi-threading of independent per-tick processes, and trained model inference could be exported via ONNX to allow deployment in lower-level environments without rewriting the training pipeline.

The training approach was constrained by available hardware. With a Ryzen 5 3600, RTX 3060 12GB, and 16GB DDR4 RAM, full training runs required approximately 100 hours. This necessitated retraining from existing checkpoints rather than training fresh models when iterating on the reward function. Whilst this shortened iteration time, it carried over undesirable behaviours from prior training runs, limiting the cleanliness of each experimental condition. Access to more capable hardware would allow full retraining per configuration and more rigorous ablation of reward function components.

The project establishes RecurrentPPO with an LSTM policy as a viable approach but does not compare it against alternative algorithms. As discussed in Section 3, DQN, QMIX, and standard PPO without recurrence are all plausible candidates for this environment. A comparative evaluation across algorithms would strengthen the evidence base and clarify whether the LSTM component provides measurable benefit over a memoryless policy. Given that each full training run requires approximately 100 hours on the hardware used, such a comparison would require either significantly more capable hardware or a reduced training budget with shorter episode limits.

Finally, the self-play curriculum did not function as intended during training, as documented in Section 9.5. A natural next step would be to resolve the snapshot saving issue and retrain from scratch with self-play active, which would expose the agent to adaptive adversarial strategies and likely produce more sophisticated multi-formation coordination.

10.2 Conclusion

This project set out to achieve three goals: a procedurally generated 2D battle environment with varied geographical features and multiple unit types, a demonstration that RecurrentPPO with an LSTM layer is a theoretically and practically superior approach to tactical combat AI, and a modular architecture allowing individual components to be extended or replaced independently.

The first objective was achieved. The environment generates varied terrain and height maps procedurally using Perlin noise and cellular automata smoothing, and supports three distinct unit types: infantry, cavalry, and archers, each with unique combat mechanics. As demonstrated in Section 8.3, the agent learned terrain-specific behaviour, most notably the halting of an archer formation on elevated ground to exploit the height range bonus, providing evidence that the geographical features are not merely cosmetic but meaningfully influence the learned policy.

The second objective was partially achieved. RecurrentPPO with an LSTM layer was shown to be theoretically well-suited to this class of problem in Section 3.3, with the recurrent architecture providing temporal memory necessary for decision-making under partial observability. The

training metrics presented in Section 8.5 confirm that learning occurred, with explained variance reaching 0.9986 and total loss reducing by three orders of magnitude. However, a practical comparative evaluation against alternative algorithms could not be conducted due to hardware constraints, as discussed in Section 10.1. The theoretical case is established; empirical comparison remains as future work.

The third objective was achieved. The system is structured around a clear class hierarchy, World, Team, Formation, Unit, with encapsulation enforced through getter and setter methods at each level. Individual components can be modified without affecting the rest of the system, as demonstrated by the curriculum stage gating, which activates terrain and line-of-sight mechanics independently without altering the core simulation loop.

The central research question, whether RecurrentPPO with an LSTM policy can learn deliberate, tactically meaningful behaviour in a mid-scale combat environment, is answered affirmatively. The agent demonstrated emergent tactical reasoning, including diagonal routing to achieve flanking effects and terrain-aware unit positioning, without these behaviours being explicitly encoded. These results establish that the methodology is viable and accessible, requiring only consumer-grade hardware and open-source tooling to produce a functional tactical combat AI.

Reflection

11.1 Reflection

This project has been as much a lesson in engineering judgement under constraint as it has been a technical exercise. The most crucial decision made during training was to retrain from existing checkpoints rather than training fresh models when iterating on the reward function. In hindsight, the practical results were limited by this choice, undesirable behaviours carried over between training runs, making it difficult to isolate the effect of individual reward changes. However, this decision was not made carelessly. Hardware limitations and submission deadlines were real constraints, and training from scratch at each iteration would have pushed the total number of training steps well beyond one billion. Given the available hardware, which was not feasible. The checkpoint approach was the right call given the circumstances, even if it was not the right call in isolation.

If starting again, the most important change would be to verify the self-play pipeline end-to-end before committing to a full training run. A simple check confirming that opponent snapshots were being saved to disk would have taken minutes to implement and would have caught the failure that silently undermined a core component of the methodology. This is the clearest example of a lesson that cannot be learned from a textbook: a system can pass 96.7% of its tests and still fail to do what it was designed to do. Rigorous testing catches functional defects; it does not guarantee that higher-level behaviours emerge as intended. The gap between a correctly functioning system and a correctly behaving system is where the most interesting engineering problems live.

The most genuinely surprising result was the explained variance reaching 0.9986, and more broadly the emergence of low-level tactical reasoning, diagonal routing to achieve flanking effects and the halting of archer formations on elevated terrain. These behaviours were not explicitly encoded anywhere in the system. They emerged from the reward signal and the structure of the observation space, which suggests the training environment was better designed than initially appreciated. In a sense, the agent found ways to exploit the system that had not been anticipated, which is both encouraging and humbling.

This project has also changed how I think about game AI more broadly. The received wisdom in the games industry appears to be that scripted behaviour trees are sufficient for tactical enemies. Having spent several months building a system that produces emergent tactical reasoning on consumer hardware with open-source tools, I find that position increasingly difficult to defend. The gap between what is being shipped and what is demonstrably achievable suggests that the barrier is not technical but organisational, a conclusion that is frustrating, but probably accurate.

Appendix

Appendix – Literature review

1. Millington, I. and Funge, J. (2009) Artificial Intelligence for Games, 2nd ed. CRC Press. 2. Isla, D. (2005) Handling Complexity in the Halo 2 AI. GDC 2005. 3. Colledanchise, M. and Ogren, P. (2018) Behavior Trees in Robotics and AI. CRC Press. 4. Sutton, R. and Barto, A. (2018) Reinforcement Learning: An Introduction, 2nd ed. MIT Press. 5. Li et al. (2025) Emergent behaviors in multiagent pursuit-evasion games. Scientific Reports. 6. Schulman, J. et al. (2017) Proximal Policy Optimization Algorithms. arXiv:1707.06347. 7. OpenAI (2019) Dota 2 with Large Scale Deep Reinforcement Learning. arXiv:1912.06680. 8. de la Fuente, N. and Vidal Guerra, D. (2024) A Comparative Study of DRL Models. KDD 2024. 9. Hochreiter, S. and Schmidhuber, J. (1997) Long Short-Term Memory. Neural Computation, 9(8). 10. Hausknecht, M. and Stone, P. (2015) Deep Recurrent Q-Learning for POMDPs. arXiv:1507.06527. 11. Raffin, A. et al. (2021) Stable-Baselines3. JMLR, 22(268). 12. Vinyals, O. et al. (2019) Grandmaster level in StarCraft II. Nature, 575. 13. Brockman, G. et al. (2016) OpenAI Gym. arXiv:1606.01540. 14. Ontanon, S. et al. (2018) The First microRTS AI Competition. IEEE ToG. 15. Andersen, P. et al. (2018) Deep RTS. arXiv:1808.05032. 16. Boron, J. et al. (2020) Developing Combat Behavior Through RL. Naval Postgraduate School. 17. Garcia et al. (2025) multi-Agent RL for NPCs. ScienceDirect. 18. Bengio, Y., Louradour, J., Collobert, R. and Weston, J. (2009) Curriculum Learning. ICML 2009. UK Ministry of Defence (2025), Renic and Schwarz (2025), UK Government (2024), UN General Assembly (2024), House of Lords (2023).

Appendix - Implementation Code Snippets

Snippet 1: Unit base class — movement

```
def move(self, destination: tuple[float, float], tick_rate: int,
         speed_modifier: float = 1.0):
    dx = destination[0] - self.x
    dy = destination[1] - self.y
    dist = math.hypot(dx, dy)
    if dist > 0:
        step = (self.speed * speed_modifier) / tick_rate
        ratio = min(step / dist, 1.0)
        self.x += dx * ratio
        self.y += dy * ratio
```

Snippet 2: Infantry — shield wall overrides

```
def attack(self, target: 'Unit'):
    if self.get_wall():
        target.set_hp(target.get_hp() - (self.damage / 2))
    else:
        target.set_hp(target.get_hp() - self.damage)
    self.is_attacking = True


def move(self, destination, tick_rate, speed_modifier=1.0):
    dx = destination[0] - self.x
    dy = destination[1] - self.y
```

```
    dist = math.hypot(dx, dy)
    if dist > 0:
        if self.get_wall():
            step = (self.speed * speed_modifier * 0.5) / tick_rate
        else:
            step = (self.speed * speed_modifier) / tick_rate
        ratio = min(step / dist, 1.0)
        self.x += dx * ratio
        self.y += dy * ratio
```

Snippet 3: Archer — damage reduction against shield wall

```
def attack(self, target: 'Unit'):
    if isinstance(target, Infantry):
        if target.get_wall():
            target.set_hp(target.get_hp() - self.damage * 0.05)
        else:
            target.set_hp(target.get_hp() - self.damage)
    else:
        target.set_hp(target.get_hp() - self.damage)
    self.is_attacking = True
```

Snippet 4: Cavalry — move while attacking

```
class Cavalry(Unit):
    def __init__(self):
        super().__init__()
        self.hp = 80
        self.damage = 7
        self.speed = 3
        self.radius = 2.0
        self.range = (self.radius * 2) + 1


    @property
    def can_move_while_attacking(self) -> bool:
        return True
```

Snippet 5: Formation — port/starboard destination calculation

```
def move(self, px, py, sx, sy, rank_spacing=DEFAULT_RANK_SPACING,
         unit_gap=DEFAULT_UNIT_GAP):
    if not self.units:
        return
    if None not in (px, py, sx, sy):
```

```
    self.set_port(px, py)
    self.set_starboard(sx, sy)
  if isinstance(self.units[0], Infantry) and self.units[0].get_wall():
    unit_gap -= 1
    rank_spacing -= 1
  radius = self.units[0].get_radius()
  unit_slot = 2 * radius + unit_gap
  dx = self.starboardx - self.portx
  dy = self.starboardy - self.porty
  length = math.hypot(dx, dy)
  if length < 1e-9:
    return
  tx, ty = dx / length, dy / length
  nx, ny = ty, -tx
  self.set_rank_width(length, unit_slot)
  units_per_rank = self.get_rank_width()
  self.set_rank_count(units_per_rank)
  num_ranks = self.get_rank_count()
  unit_idx = 0
  for rank in range(num_ranks):
    count_this_rank = min(units_per_rank, len(self.units) - unit_idx)
    rank_span = (count_this_rank - 1) * unit_slot
    start_offset = (length - rank_span) / 2
    for i in range(count_this_rank):
      along = start_offset + i * unit_slot
      depth = rank * rank_spacing
      x = self.portx + along * tx + depth * nx
      y = self.porty + along * ty + depth * ny
      self.units[unit_idx].set_destination(x, y)
      unit_idx += 1
```

Snippet 6: Team

```
class Team:
  def __init__(self, team_id: bool):
    self.team_id = team_id
    self.formations: list[Formation] = []


  def add_formation(self, formation: Formation):
    self.formations.append(formation)
```

```
    def get_formations(self) -> list[Formation]:
        return self.formations


    def get_all_units(self) -> list[Unit]:
        return [u for f in self.formations for u in f.get_units()]


    def get_living_units(self) -> list[Unit]:
        return [u for f in self.formations for u in f.get_living_units()]


    def is_defeated(self) -> bool:
        return len(self.get_living_units()) == 0
```

Snippet 7: Constants and army presets (excerpt)

```
TERRAIN_SPEED = {
    Terrain.PLAINS: 1.0,
    Terrain.WATER:  0.3,
    Terrain.FOREST: 0.7,
}
FOREST_DETECTION_RANGE = 50
HEIGHT_RANGE_BONUS_PER_LEVEL = 1
SIGHT_RANGE = 100
DEFAULT_RANK_SPACING = 2.0
DEFAULT_UNIT_GAP = 2.0


ARMY_PRESETS = [
    {"name": "Balanced",
     "formations": [
        {"type": "infantry", "count": 30},
        {"type": "archer",   "count": 20},
        {"type": "cavalry",  "count": 15}]},
    {"name": "Cavalry hammer",
     "formations": [
        {"type": "infantry", "count": 25},
        {"type": "cavalry",  "count": 30}]},
    # ... 10 presets total
]
```

Snippet 8: Team population

```
def populate_team(self, team: Team):
    preset = ARMY_PRESETS[np.random.randint(len(ARMY_PRESETS))]
    num_formations = len(preset["formations"])
    margin = 5
    available_height = self.height - 2 * margin
    slot_height = available_height / num_formations


    if not team.team_id:
        x_start, x_end = 80, 100
    else:
        x_start, x_end = self.width - 100, self.width - 80


    for i, spec in enumerate(preset["formations"]):
        UnitClass = UNIT_TYPES[spec["type"]]
        units = [UnitClass() for _ in range(spec["count"])]
        for unit in units:
            unit.set_team(team.team_id)


        y_top = margin + i * slot_height
        y_bottom = margin + (i + 1) * slot_height


        formation = Formation(units)
        formation.move(x_start, y_top, x_end, y_bottom)
        team.add_formation(formation)
        formation.move(x_start, y_top, x_end, y_bottom)
        for unit in formation.get_units():
            x, y = unit.get_destination()
            unit.set_x(x)
            unit.set_y(y)


    self.add_team(team)
```

Snippet 9: Formation chosion Measurment

```
def measure_formation_cohesion(self) -> float:
        """
        How close a formation's living units are to a perfect line.
```

```
Returns float in [0, 1].  1 = perfect line, 0 = total disorder.

Two components, equally weighted:
- perp_score:    mean perpendicular distance from the line
- spacing_score: how evenly units are distributed along it

Sorted projection comparison makes this robust to unit deaths.
"""
living = self.get_living_units()
n = len(living)
if n <= 1:
    return 1.0

px, py = self.get_port()
sx, sy = self.get_starboard()
ldx = sx - px
ldy = sy - py
line_len = math.hypot(ldx, ldy)

if line_len < 1e-9:
    return 0.5

# Unit vectors: along line and normal
along_x = ldx / line_len
along_y = ldy / line_len
norm_x = -along_y
norm_y =  along_x

perp_distances = []
projections = []

for u in living:
    dx = u.x - px
    dy = u.y - py
    t = (dx * along_x + dy * along_y) / line_len   # [0,1] ideally
    perp = abs(dx * norm_x + dy * norm_y)
    projections.append(t)
    perp_distances.append(perp)
```

```
    # ── Perpendicular score ──────────────────────────────────────
    mean_perp = sum(perp_distances) / n

    unit_slot = 2 * living[0].get_radius() + DEFAULT_UNIT_GAP
    expected_len = n * unit_slot
    perp_score = max(0.0, 1.0 - mean_perp / (expected_len * 0.5))

    # ── Spacing score ────────────────────────────────────────────
    projections.sort()
    ideal = [i / (n - 1) for i in range(n)]
    spacing_error = sum(abs(projections[i] - ideal[i]) for i in range(n)) / n
    # An average error of 0.5 (half the line) ⇒ score ~ 0
    spacing_score = max(0.0, 1.0 - spacing_error * 2)

    return (perp_score + spacing_score) / 2
```

Snippet 10: Perlin noise generation

```
def _generate_perlin_noise(self, scale=50.0, octaves=6,
                           persistence=0.5, lacunarity=2.0):
    noise_map = np.zeros((self.height, self.width))
    amplitude, frequency, max_value = 1.0, 1.0, 0.0
    for octave in range(octaves):
        octave_noise = self._generate_smooth_noise(scale / frequency)
        noise_map += octave_noise * amplitude
        max_value += amplitude
        amplitude *= persistence
        frequency *= lacunarity
    return noise_map / max_value
```

Snippet 11: Cellular automata smoothing

```
def _apply_cellular_automata(self, grid, iterations=3):
    smoothed = np.copy(grid)
    kernel = np.ones((3, 3), dtype=int)
    for _ in range(iterations):
        unique_vals = np.unique(smoothed)
        counts = np.zeros((len(unique_vals), self.height, self.width))
        for i, val in enumerate(unique_vals):
            mask = (smoothed == val).astype(int)
            counts[i] = convolve2d(mask, kernel, mode='same',
```

```
                    boundary='fill')
    winner_indices = np.argmax(counts, axis=0)
    smoothed = unique_vals[winner_indices]
  return smoothed
```

Snippet 12: Terrain map generation

```
def _generate_terrain_map(self):
  noise = self._generate_perlin_noise(scale=30.0, octaves=6,
                                      persistence=0.5)
  terrain_map = np.zeros((self.height, self.width), dtype=int)
  terrain_map[noise < 0.15] = Terrain.WATER
  terrain_map[(noise >= 0.15) & (noise < 0.85)] = Terrain.PLAINS
  terrain_map[noise >= 0.85] = Terrain.FOREST
  return self._apply_cellular_automata(terrain_map, iterations=5)


def _generate_height_map(self):
  noise = self._generate_perlin_noise(scale=25.0, octaves=4,
                                      persistence=0.6)
  height_map = np.clip((noise * 5).astype(int), 0, 5)
  height_map = self._apply_cellular_automata(height_map, iterations=3)
  height_map[self.terrain_map == Terrain.WATER] = 0
  return height_map
```

Snippet 13: Combat phase — cKDTree targeting

```
def _combat_phase(self, living):
  for u in living:
    u.is_attacking = False
  for team in self.teams:
    enemy_units = []
    for et in self.teams:
      if et.team_id == team.team_id:
        continue
      for f in et.get_formations():
        enemy_units.extend(f.get_living_units())
    if not enemy_units:
      continue
    enemy_pos = np.array([[u.x, u.y] for u in enemy_units])
    enemy_tree = cKDTree(enemy_pos)
    attackers = []
    for f in team.get_formations():
```

```
            attackers.extend(f.get_living_units())
        if not attackers:
            continue
        attacker_pos = np.array([[u.x, u.y] for u in attackers])
        dists, indices = enemy_tree.query(attacker_pos)
        for i, attacker in enumerate(attackers):
            if attacker.is_dead():
                continue
            target = enemy_units[indices[i]]
            if target.is_dead():
                continue
            dist = dists[i]
            eff_range = self.get_effective_range_against(attacker, target)
            if dist <= eff_range and self.can_target(attacker, target):
                attacker.attack(target)
            elif dist <= SIGHT_RANGE and self.can_see(attacker, target):
                attacker.set_destination(target.x, target.y)
```

Snippet 14: Collision resolution

```
def _resolve_collisions(self, living, iterations=3):
    if len(living) < 2:
        return
    max_radius = max(u.get_radius() for u in living)
    query_radius = max_radius * 2
    for _ in range(iterations):
        positions = np.array([[u.x, u.y] for u in living])
        tree = cKDTree(positions)
        pairs = tree.query_pairs(query_radius)
        if not pairs:
            break
        resolved_any = False
        for i, j in pairs:
            a, b = living[i], living[j]
            dx, dy = b.x - a.x, b.y - a.y
            dist = math.hypot(dx, dy)
            min_dist = a.radius + b.radius
            if dist >= min_dist:
                continue
            resolved_any = True
```

```
            overlap = min_dist - dist
            if dist > 0:
                nx, ny = dx / dist, dy / dist
            else:
                nx, ny = 1.0, 0.0
            push = overlap / 2
            a.x -= nx * push
            a.y -= ny * push
            b.x += nx * push
            b.y += ny * push
        for u in living:
            u.x = max(0, min(self.width - 1, u.x))
            u.y = max(0, min(self.height - 1, u.y))
        if not resolved_any:
            break
```

Snippet 15: Bresenham's line algorithm for line of sight

```
def has_line_of_sight(self, a, b):
    if self.curriculum_stage < 3:
        return True
    ax, ay = self._clamp_grid(a.x, a.y)
    bx, by = self._clamp_grid(b.x, b.y)
    h_a = self.height_map[ay, ax]
    h_b = self.height_map[by, bx]
    max_endpoint = max(h_a, h_b)
    for gx, gy in self._bresenham(ax, ay, bx, by):
        if (gx, gy) == (ax, ay) or (gx, gy) == (bx, by):
            continue
        if self.height_map[gy, gx] > max_endpoint:
            return False
    return True


@staticmethod
def _bresenham(x0, y0, x1, y1):
    dx, dy = abs(x1 - x0), abs(y1 - y0)
    sx = 1 if x0 < x1 else -1
    sy = 1 if y0 < y1 else -1
    err = dx - dy
    while True:
```

```
        yield (x0, y0)
        if x0 == x1 and y0 == y1:
            break
        e2 = 2 * err
        if e2 > -dy:
            err -= dy; x0 += sx
        if e2 < dx:
            err += dx; y0 += sy
```

Snippet 16: Reward function

```
def _compute_reward(self):
    reward = 0.0
    current_friendly_hp = self._total_hp(self.friendly_team)
    current_enemy_hp = self._total_hp(self.enemy_team)
    friendly_hp_lost = self.prev_friendly_hp - current_friendly_hp
    enemy_hp_lost = self.prev_enemy_hp - current_enemy_hp
    hp_scale = max(self.prev_friendly_hp + self.prev_enemy_hp, 1)
    reward += enemy_hp_lost / hp_scale * 0.5
    reward -= friendly_hp_lost / hp_scale * 0.1
    self.prev_friendly_hp = current_friendly_hp
    self.prev_enemy_hp = current_enemy_hp


    avg_distance = self._avg_distance_to_enemy()
    max_distance = self.world.get_diagonal_length()
    distance_closed = self.prev_avg_distance - avg_distance
    reward += distance_closed / max_distance * 2.0
    self.prev_avg_distance = avg_distance


    for formation in self.friendly_team.get_formations():
        living = formation.get_living_units()
        if not living:
            continue
        cohesion = formation.measure_formation_cohesion()
        cohesion_reward = max(0.0, 1.0 - abs(cohesion - 0.7) / 0.3)
        reward += cohesion_reward * 0.5
        n = len(living)
        unit_slot = 2 * living[0].get_radius() + model.DEFAULT_UNIT_GAP
        expected_len = n * unit_slot
        actual_len = formation.get_diagonal_length()
```

```
        if actual_len > expected_len * 2.0:
            reward -= 1


    if self.enemy_team.is_defeated():
        reward += 10.0
    elif self.friendly_team.is_defeated():
        reward -= 10.0
    return reward
```

Snippet 17: Observation encoding (per formation)

```
def _encode_team_formations(self, obs, formations, offset, mirror_x=False):
    for i, formation in enumerate(formations):
        if i >= MAX_FORMATIONS:
            break
        living = formation.get_living_units()
        if not living:
            continue
        base = offset + (i * FEATURES_PER_FORMATION)
        unit_type_name = UNIT_CLASS_TO_TYPE.get(type(living[0]), "infantry")
        xs = [u.get_x() for u in living]
        ys = [u.get_y() for u in living]
        avg_x = sum(xs) / len(xs)
        avg_y = sum(ys) / len(ys)
        min_x, max_x = min(xs), max(xs)
        min_y, max_y = min(ys), max(ys)
        if mirror_x:
            avg_x = self.world.width - avg_x
            min_x, max_x = self.world.width - max_x, self.world.width - min_x
        obs[base + 0] = 1.0
        obs[base + 1] = UNIT_TYPE_ENCODING.get(unit_type_name, 0.0)
        obs[base + 2] = min(len(living) / 50.0, 1.0)
        obs[base + 3] = min_x / self.world.width
        obs[base + 4] = max_x / self.world.width
        obs[base + 5] = avg_y / self.world.height
        obs[base + 6] = self.world.get_height_at(avg_x, avg_y) / 5.0
        obs[base + 7] = self.world.get_terrain_at(avg_x, avg_y) / 2.0
        if unit_type_name == "infantry":
            obs[base + 8] = 1.0 if living[0].get_wall() else 0.5
        else:
```

```
            obs[base + 8] = 0.0
        obs[base + 9] = max_y / self.world.height
        obs[base + 10] = min_y / self.world.height
        obs[base + 11] = avg_x / self.world.width
        obs[base + 12] = formation.get_rank_count() / self.world.get_diagonal_length()
        obs[base + 13] = formation.get_rank_width() / self.world.get_diagonal_length()
```

Snippet 18: Self-play mirrored observation

```
def _build_opponent_obs(self):
    obs = np.zeros(OBS_SIZE, dtype=np.float32)
    self._encode_team_formations(
        obs, self.enemy_team.get_formations(),
        offset=0, mirror_x=True)
    self._encode_team_formations(
        obs, self.friendly_team.get_formations(),
        offset=MAX_FORMATIONS * FEATURES_PER_FORMATION,
        mirror_x=True)
    self._encode_global_features(
        obs, offset=MAX_FORMATIONS * FEATURES_PER_FORMATION * 2)
    return obs
```

Appendix — Test Code Snippets

Snippet 19: TestUnitStats

```
class TestUnitStats:
    def test_infantry_stats(self):
        u = model.Infantry()
        assert u.hp == 100
        assert u.damage == 10
        assert u.speed == 1
        assert u.radius == 1.0
        assert u.range == 4


    def test_archer_stats(self):
        u = model.Archer()
        assert u.hp == 80
        assert u.damage == 5
        assert u.speed == 1.2
        assert u.radius == 0.5
```

```
        assert u.range == 51


    def test_cavalry_stats(self):
        u = model.Cavalry()
        assert u.hp == 80
        assert u.damage == 7
        assert u.speed == 3
        assert u.radius == 2.0
        assert u.range == 5
```

Snippet 20: TestUnitDeath

```
class TestUnitDeath:
    def test_unit_dies_at_zero_hp(self):
        u = model.Infantry()
        u.set_hp(0)
        assert u.is_dead()


    def test_unit_dies_below_zero_hp(self):
        u = model.Infantry()
        u.set_hp(-10)
        assert u.is_dead()


    def test_unit_alive_at_one_hp(self):
        u = model.Infantry()
        u.set_hp(1)
        assert not u.is_dead()
```

Snippet 21: TestUnitMovement

```
class TestUnitMovement:
    def test_unit_moves_toward_destination(self):
        u = model.Infantry()
        u.set_x(0.0); u.set_y(0.0)
        u.set_destination(100.0, 0.0)
        u.on_tick(tick_rate=1, speed_modifier=1.0)
        assert u.get_x() > 0.0


    def test_unit_does_not_overshoot(self):
        u = model.Infantry()
        u.set_x(0.0); u.set_y(0.0)
```

```
        u.set_destination(0.5, 0.0)
        u.on_tick(tick_rate=1, speed_modifier=1.0)
        assert u.get_x() <= 0.5

    def test_cavalry_faster_than_infantry(self):
        inf = model.Infantry()
        inf.set_x(0.0); inf.set_y(0.0)
        inf.set_destination(100.0, 0.0)
        cav = model.Cavalry()
        cav.set_x(0.0); cav.set_y(0.0)
        cav.set_destination(100.0, 0.0)
        inf.on_tick(tick_rate=10)
        cav.on_tick(tick_rate=10)
        assert cav.get_x() > inf.get_x()

    def test_attacking_unit_stops_moving(self):
        u = model.Infantry()
        u.set_x(0.0); u.set_y(0.0)
        u.set_destination(100.0, 0.0)
        u.is_attacking = True
        u.on_tick(tick_rate=10)
        assert u.get_x() == 0.0

    def test_cavalry_moves_while_attacking(self):
        u = model.Cavalry()
        u.set_x(0.0); u.set_y(0.0)
        u.set_destination(100.0, 0.0)
        u.is_attacking = True
        u.on_tick(tick_rate=10)
        assert u.get_x() > 0.0
```

Snippet 22: TestShieldWall

```
class TestShieldWall:
    def test_shield_wall_halves_outgoing_damage(self):
        attacker = model.Infantry()
        attacker.set_wall(True)
        target = model.Infantry()
        initial_hp = target.get_hp()
        attacker.attack(target)
```

```
    damage_dealt = initial_hp - target.get_hp()
    assert damage_dealt == 5

def test_no_wall_full_damage(self):
    attacker = model.Infantry()
    attacker.set_wall(False)
    target = model.Infantry()
    initial_hp = target.get_hp()
    attacker.attack(target)
    damage_dealt = initial_hp - target.get_hp()
    assert damage_dealt == 10

def test_archer_vs_shield_wall_95_percent_reduction(self):
    archer = model.Archer()
    target = model.Infantry()
    target.set_wall(True)
    initial_hp = target.get_hp()
    archer.attack(target)
    damage_dealt = initial_hp - target.get_hp()
    expected = 5 * 0.05
    assert abs(damage_dealt - expected) < 0.01

def test_archer_vs_no_wall_full_damage(self):
    archer = model.Archer()
    target = model.Infantry()
    target.set_wall(False)
    initial_hp = target.get_hp()
    archer.attack(target)
    damage_dealt = initial_hp - target.get_hp()
    assert damage_dealt == 5

def test_archer_vs_non_infantry_full_damage(self):
    archer = model.Archer()
    target = model.Cavalry()
    initial_hp = target.get_hp()
    archer.attack(target)
    damage_dealt = initial_hp - target.get_hp()
    assert damage_dealt == 5
```

```
  def test_shield_wall_halves_movement_speed(self):
    u1 = model.Infantry()
    u1.set_x(0.0); u1.set_y(0.0)
    u1.set_destination(100.0, 0.0)
    u1.set_wall(False)
    u2 = model.Infantry()
    u2.set_x(0.0); u2.set_y(0.0)
    u2.set_destination(100.0, 0.0)
    u2.set_wall(True)
    u1.on_tick(tick_rate=10)
    u2.on_tick(tick_rate=10)
    assert abs(u2.get_x() - u1.get_x() / 2) < 0.01
```

Snippet 23: TestFormation

```
class TestFormation:
  def test_units_receive_destinations(self):
    units = [model.Infantry() for _ in range(10)]
    f = model.Formation(units)
    f.move(0, 0, 50, 0)
    for u in units:
      assert u.get_destination() is not None

  def test_destinations_within_bounds(self):
    units = [model.Infantry() for _ in range(30)]
    f = model.Formation(units)
    f.move(10, 10, 90, 10)
    for u in units:
      dx, dy = u.get_destination()
      assert dx >= 0
      assert dy >= 0

  def test_rank_count_increases_with_depth(self):
    units = [model.Infantry() for _ in range(30)]
    f = model.Formation(units)
    f.move(0, 0, 10, 0)
    assert f.get_rank_count() > 1
```

```
    def test_single_unit_formation(self):
        units = [model.Infantry()]
        f = model.Formation(units)
        f.move(0, 0, 50, 0)
        assert units[0].get_destination() is not None

    def test_empty_formation_no_crash(self):
        f = model.Formation([])
        f.move(0, 0, 50, 0)

    def test_living_units_excludes_dead(self):
        units = [model.Infantry() for _ in range(5)]
        units[0].set_hp(0)
        units[2].set_hp(0)
        f = model.Formation(units)
        assert len(f.get_living_units()) == 3
        assert len(f.get_units()) == 5
```

Snippet 24: TestTeam

```
class TestTeam:
    def test_team_defeated_when_all_dead(self):
        team = model.Team(team_id=False)
        units = [model.Infantry() for _ in range(5)]
        for u in units:
            u.set_hp(0)
        team.add_formation(model.Formation(units))
        assert team.is_defeated()

    def test_team_not_defeated_with_living(self):
        team = model.Team(team_id=False)
        units = [model.Infantry() for _ in range(5)]
        units[0].set_hp(0)
        team.add_formation(model.Formation(units))
        assert not team.is_defeated()

    def test_get_living_units_across_formations(self):
        team = model.Team(team_id=False)
        f1 = model.Formation([model.Infantry() for _ in range(3)])
```

```
    f2 = model.Formation([model.Archer() for _ in range(4)])
    f1.get_units()[0].set_hp(0)
    team.add_formation(f1)
    team.add_formation(f2)
    assert len(team.get_living_units()) == 6
```

Snippet 25: TestTerrainGeneration

```
class TestTerrainGeneration:
  @pytest.fixture
  def world(self):
    return model.World(100, 100, tick_rate=10, curriculum_stage=3)

  def test_terrain_map_shape(self, world):
    assert world.terrain_map.shape == (100, 100)

  def test_height_map_shape(self, world):
    assert world.height_map.shape == (100, 100)

  def test_terrain_values_valid(self, world):
    unique = set(np.unique(world.terrain_map))
    valid = {model.Terrain.PLAINS, model.Terrain.WATER, model.Terrain.FOREST}
    assert unique.issubset(valid)

  def test_height_values_in_range(self, world):
    assert world.height_map.min() >= 0
    assert world.height_map.max() <= 5

  def test_water_tiles_height_zero(self, world):
    water_mask = world.terrain_map == model.Terrain.WATER
    if water_mask.any():
      assert np.all(world.height_map[water_mask] == 0)

  def test_terrain_deterministic_with_seed(self):
    np.random.seed(42)
    w1 = model.World(50, 50, tick_rate=10)
    np.random.seed(42)
    w2 = model.World(50, 50, tick_rate=10)
    assert np.array_equal(w1.terrain_map, w2.terrain_map)
```

```
    assert np.array_equal(w1.height_map, w2.height_map)
```

Snippet 26: TestSpeedModifiers

```
class TestSpeedModifiers:
  def test_stage0_always_returns_1(self):
    world = model.World(50, 50, tick_rate=10, curriculum_stage=0)
    for y in range(50):
      for x in range(50):
        assert world.get_speed_modifier(x, y) == 1.0


  def test_stage1_water_slows(self):
    world = model.World(100, 100, tick_rate=10, curriculum_stage=1)
    water_positions = np.argwhere(world.terrain_map == model.Terrain.WATER)
    if len(water_positions) > 0:
      wy, wx = water_positions[0]
      assert world.get_speed_modifier(wx, wy) == 0.3


  def test_stage1_plains_normal(self):
    world = model.World(100, 100, tick_rate=10, curriculum_stage=1)
    plains_positions = np.argwhere(world.terrain_map == model.Terrain.PLAINS)
    if len(plains_positions) > 0:
      py, px = plains_positions[0]
      assert world.get_speed_modifier(px, py) == 1.0
```

Snippet 27: TestConcealment

```
class TestConcealment:
  def test_stage0_no_concealment(self):
    world = model.World(100, 100, tick_rate=10, curriculum_stage=0)
    u = model.Infantry()
    assert not world.is_concealed(u)


  def test_stage2_forest_conceals(self):
    world = model.World(100, 100, tick_rate=10, curriculum_stage=2)
    forest_positions = np.argwhere(world.terrain_map == model.Terrain.FOREST)
    if len(forest_positions) > 0:
      fy, fx = forest_positions[0]
      u = model.Infantry()
      u.set_x(float(fx)); u.set_y(float(fy))
      assert world.is_concealed(u)
```

```
  def test_concealed_unit_detected_within_range(self):
    world = model.World(100, 100, tick_rate=10, curriculum_stage=2)
    forest_positions = np.argwhere(world.terrain_map == model.Terrain.FOREST)
    if len(forest_positions) > 0:
      fy, fx = forest_positions[0]
      target = model.Infantry()
      target.set_x(float(fx)); target.set_y(float(fy))
      observer = model.Infantry()
      observer.set_x(float(fx) + 10); observer.set_y(float(fy))
      assert world.can_detect(observer, target)

  def test_concealed_unit_not_detected_outside_range(self):
    world = model.World(200, 200, tick_rate=10, curriculum_stage=2)
    forest_positions = np.argwhere(world.terrain_map == model.Terrain.FOREST)
    if len(forest_positions) > 0:
      fy, fx = forest_positions[0]
      target = model.Infantry()
      target.set_x(float(fx)); target.set_y(float(fy))
      observer = model.Infantry()
      observer.set_x(float(fx) + 60); observer.set_y(float(fy))
      assert not world.can_detect(observer, target)
```

Snippet 28: TestLineOfSight

```
class TestLineOfSight:
  def test_los_always_true_below_stage3(self):
    world = model.World(50, 50, tick_rate=10, curriculum_stage=2)
    a = model.Infantry(); a.set_x(0); a.set_y(0)
    b = model.Infantry(); b.set_x(49); b.set_y(49)
    assert world.has_line_of_sight(a, b)

  def test_los_blocked_by_peak(self):
    world = model.World(50, 50, tick_rate=10, curriculum_stage=3)
    world.height_map[:] = 1
    world.height_map[25, 25] = 5
    a = model.Infantry(); a.set_x(0); a.set_y(25)
    b = model.Infantry(); b.set_x(49); b.set_y(25)
    assert not world.has_line_of_sight(a, b)
```

```
  def test_los_not_blocked_by_equal_height(self):
    world = model.World(50, 50, tick_rate=10, curriculum_stage=3)
    world.height_map[:] = 3
    a = model.Infantry(); a.set_x(0); a.set_y(25)
    b = model.Infantry(); b.set_x(49); b.set_y(25)
    assert world.has_line_of_sight(a, b)

  def test_los_high_ground_sees_over_low_peak(self):
    world = model.World(50, 50, tick_rate=10, curriculum_stage=3)
    world.height_map[:] = 1
    world.height_map[25, 25] = 3
    world.height_map[25, 0] = 4
    world.height_map[25, 49] = 4
    a = model.Infantry(); a.set_x(0); a.set_y(25)
    b = model.Infantry(); b.set_x(49); b.set_y(25)
    assert world.has_line_of_sight(a, b)
```

Snippet 29: TestHeightRangeBonus

```
class TestHeightRangeBonus:
  def test_higher_ground_increases_range(self):
    world = model.World(50, 50, tick_rate=10, curriculum_stage=3)
    world.height_map[:] = 1
    world.height_map[10, 10] = 5
    attacker = model.Archer()
    attacker.set_x(10); attacker.set_y(10)
    target = model.Infantry()
    target.set_x(10); target.set_y(20)
    eff_range = world.get_effective_range_against(attacker, target)
    assert eff_range == float(attacker.range) + 4

  def test_lower_ground_decreases_range(self):
    world = model.World(50, 50, tick_rate=10, curriculum_stage=3)
    world.height_map[:] = 5
    world.height_map[10, 10] = 1
    attacker = model.Archer()
    attacker.set_x(10); attacker.set_y(10)
    target = model.Infantry()
    target.set_x(10); target.set_y(20)
```

```
        eff_range = world.get_effective_range_against(attacker, target)
        assert eff_range == float(attacker.range) - 4


    def test_no_range_bonus_before_stage3(self):
        world = model.World(50, 50, tick_rate=10, curriculum_stage=2)
        world.height_map[:] = 1
        world.height_map[10, 10] = 5
        attacker = model.Archer()
        attacker.set_x(10); attacker.set_y(10)
        target = model.Infantry()
        target.set_x(10); target.set_y(20)
        assert world.get_effective_range_against(attacker, target) == float(attacker.range)
```

Snippet 30: TestCombat

```
class TestCombat:
    def test_units_in_range_deal_damage(self):
        world = model.World(100, 100, tick_rate=10, curriculum_stage=0)
        team_a = model.Team(team_id=False)
        team_b = model.Team(team_id=True)
        attacker = model.Infantry()
        attacker.set_team(False); attacker.set_x(50); attacker.set_y(50)
        target = model.Infantry()
        target.set_team(True); target.set_x(52); target.set_y(50)
        team_a.add_formation(model.Formation([attacker]))
        team_b.add_formation(model.Formation([target]))
        world.add_team(team_a)
        world.add_team(team_b)
        initial_hp = target.get_hp()
        world._combat_phase(world.get_living_units())
        assert target.get_hp() < initial_hp


    def test_units_out_of_range_no_damage(self):
        world = model.World(300, 300, tick_rate=10, curriculum_stage=0)
        team_a = model.Team(team_id=False)
        team_b = model.Team(team_id=True)
        attacker = model.Infantry()
        attacker.set_team(False); attacker.set_x(50); attacker.set_y(50)
        target = model.Infantry()
        target.set_team(True); target.set_x(200); target.set_y(50)
```

```
        team_a.add_formation(model.Formation([attacker]))
        team_b.add_formation(model.Formation([target]))
        world.add_team(team_a)
        world.add_team(team_b)
        initial_hp = target.get_hp()
        world._combat_phase(world.get_living_units())
        assert target.get_hp() == initial_hp

    def test_dead_units_do_not_attack(self):
        world = model.World(100, 100, tick_rate=10, curriculum_stage=0)
        team_a = model.Team(team_id=False)
        team_b = model.Team(team_id=True)
        attacker = model.Infantry()
        attacker.set_team(False); attacker.set_x(50); attacker.set_y(50)
        attacker.set_hp(0)
        target = model.Infantry()
        target.set_team(True); target.set_x(52); target.set_y(50)
        team_a.add_formation(model.Formation([attacker]))
        team_b.add_formation(model.Formation([target]))
        world.add_team(team_a)
        world.add_team(team_b)
        initial_hp = target.get_hp()
        world._combat_phase(world.get_living_units())
        assert target.get_hp() == initial_hp
```

Snippet 31: TestCollisionResolution

```
class TestCollisionResolution:
    def test_overlapping_units_separated(self):
        world = model.World(100, 100, tick_rate=10, curriculum_stage=0)
        u1 = model.Infantry(); u1.set_x(50); u1.set_y(50)
        u2 = model.Infantry(); u2.set_x(50); u2.set_y(50)
        world._resolve_collisions([u1, u2])
        dist = math.hypot(u1.get_x() - u2.get_x(), u1.get_y() - u2.get_y())
        min_dist = u1.radius + u2.radius
        assert dist >= min_dist - 0.01

    def test_non_overlapping_units_unchanged(self):
        world = model.World(100, 100, tick_rate=10, curriculum_stage=0)
        u1 = model.Infantry(); u1.set_x(10); u1.set_y(50)
```

```
    u2 = model.Infantry(); u2.set_x(90); u2.set_y(50)
    x1_before, x2_before = u1.get_x(), u2.get_x()
    world._resolve_collisions([u1, u2])
    assert u1.get_x() == x1_before
    assert u2.get_x() == x2_before

  def test_units_clamped_to_world_bounds(self):
    world = model.World(100, 100, tick_rate=10, curriculum_stage=0)
    u1 = model.Infantry(); u1.set_x(0); u1.set_y(0)
    u2 = model.Infantry(); u2.set_x(0); u2.set_y(0)
    world._resolve_collisions([u1, u2])
    for u in [u1, u2]:
      assert u.get_x() >= 0 and u.get_x() <= 99
      assert u.get_y() >= 0 and u.get_y() <= 99
```

Snippet 32: TestCohesion

```
class TestCohesion:
  def test_cohesion_in_valid_range(self):
    units = [model.Infantry() for _ in range(20)]
    f = model.Formation(units)
    f.move(0, 0, 50, 0)
    for u in units:
      dx, dy = u.get_destination()
      u.set_x(dx); u.set_y(dy)
    cohesion = f.measure_formation_cohesion()
    assert 0.0 <= cohesion <= 1.0

  def test_perfect_line_high_cohesion(self):
    units = [model.Infantry() for _ in range(10)]
    f = model.Formation(units)
    f.move(0, 50, 50, 50)
    for u in units:
      dx, dy = u.get_destination()
      u.set_x(dx); u.set_y(dy)
    cohesion = f.measure_formation_cohesion()
    assert cohesion > 0.8

  def test_single_unit_cohesion_is_one(self):
    units = [model.Infantry()]
```

```
        f = model.Formation(units)
        f.move(0, 0, 50, 0)
        assert f.measure_formation_cohesion() == 1.0

    def test_scattered_units_low_cohesion(self):
        units = [model.Infantry() for _ in range(10)]
        f = model.Formation(units)
        f.move(0, 0, 50, 0)
        np.random.seed(123)
        for u in units:
            u.set_x(np.random.uniform(0, 200))
            u.set_y(np.random.uniform(0, 200))
        cohesion = f.measure_formation_cohesion()
        assert cohesion < 0.5
```

Snippet 33: TestPopulation

```
class TestPopulation:
    def test_populated_team_has_formations(self):
        world = model.World(300, 300, tick_rate=10)
        team = model.Team(team_id=False)
        world.populate_team(team)
        assert len(team.get_formations()) > 0

    def test_populated_team_has_living_units(self):
        world = model.World(300, 300, tick_rate=10)
        team = model.Team(team_id=False)
        world.populate_team(team)
        assert len(team.get_living_units()) > 0

    def test_all_units_have_correct_team_id(self):
        world = model.World(300, 300, tick_rate=10)
        team = model.Team(team_id=True)
        world.populate_team(team)
        for u in team.get_all_units():
            assert u.get_team() == True

    def test_units_within_world_bounds(self):
        world = model.World(300, 300, tick_rate=10)
```

```
    team = model.Team(team_id=False)
    world.populate_team(team)
    for u in team.get_all_units():
      assert 0 <= u.get_x() <= 300
      assert 0 <= u.get_y() <= 300


  def test_max_six_formations(self):
    for preset in model.ARMY_PRESETS:
      assert len(preset["formations"]) <= 6
```

Snippet 34: TestTick

```
class TestTick:
  def test_tick_increments_count(self):
    world = model.World(300, 300, tick_rate=10, curriculum_stage=0)
    team_a = model.Team(team_id=False)
    team_b = model.Team(team_id=True)
    world.populate_team(team_a)
    world.populate_team(team_b)
    assert world.get_tick_count() == 0
    world.tick()
    assert world.get_tick_count() == 1


  def test_100_ticks_no_crash(self):
    world = model.World(300, 300, tick_rate=10, curriculum_stage=3)
    team_a = model.Team(team_id=False)
    team_b = model.Team(team_id=True)
    world.populate_team(team_a)
    world.populate_team(team_b)
    for _ in range(100):
      world.tick()


  def test_battle_eventually_ends(self):
    world = model.World(300, 300, tick_rate=10, curriculum_stage=0)
    team_a = model.Team(team_id=False)
    team_b = model.Team(team_id=True)
    world.populate_team(team_a)
    world.populate_team(team_b)
    for f in team_a.get_formations():
      f.move(140, 50, 160, 250)
```

```
    for f in team_b.get_formations():
      f.move(140, 50, 160, 250)
    initial_hp = sum(u.get_hp() for u in world.get_all_units())
    for _ in range(2000):
      world.tick()
      if team_a.is_defeated() or team_b.is_defeated():
        break
    final_hp = sum(u.get_hp() for u in world.get_all_units())
    assert final_hp < initial_hp
```

Snippet 35: TestBresenham

```
class TestBresenham:
  def test_horizontal_line(self):
    points = list(model.World._bresenham(0, 0, 5, 0))
    assert points == [(0, 0), (1, 0), (2, 0), (3, 0), (4, 0), (5, 0)]


  def test_vertical_line(self):
    points = list(model.World._bresenham(0, 0, 0, 3))
    assert points == [(0, 0), (0, 1), (0, 2), (0, 3)]


  def test_single_point(self):
    points = list(model.World._bresenham(5, 5, 5, 5))
    assert points == [(5, 5)]


  def test_includes_start_and_end(self):
    points = list(model.World._bresenham(0, 0, 10, 7))
    assert points[0] == (0, 0)
    assert points[-1] == (10, 7)
```

Snippet 36: Visual Demo Scenarios

```
TICK_RATE = 30
FRAME_MS = 33


def flat_world(width=120, height=60, seed=0):
  np.random.seed(seed)
  w = World(width, height, tick_rate=TICK_RATE, curriculum_stage=0)
  w.terrain_map[:] = Terrain.PLAINS
  w.height_map[:] = 0
  return w
```

```
def scenario_move():
    world = flat_world(width=120, height=60)
    team = Team(team_id=False)
    formation = add_formation(team, Infantry, count=12,
                              px=10, py=25, sx=10, sy=45)
    world.add_team(team)
    def step(t):
        if t == 60:
            formation.move(px=100, py=25, sx=100, sy=45)
    run(root, world, view, step_hook=step, duration_ticks=600)


def scenario_wall():
    world = flat_world(width=120, height=60)
    red = Team(team_id=False)
    red_form = add_formation(red, Infantry, count=10,
                             px=40, py=15, sx=40, sy=45)
    blue = Team(team_id=True)
    add_formation(blue, Infantry, count=10, px=60, py=15, sx=60, sy=45)
    add_formation(blue, Archer, count=8, px=80, py=20, sx=80, sy=40)
    world.add_team(red)
    world.add_team(blue)
    def step(t):
        if t == 1:
            for u in red_form.get_units():
                u.set_wall(True)
    run(root, world, view, step_hook=step, duration_ticks=1200)


def scenario_cavalry():
    world = flat_world(width=160, height=60)
    red = Team(team_id=False)
    cav_form = add_formation(red, Cavalry, count=8,
                             px=20, py=25, sx=20, sy=35)
    blue = Team(team_id=True)
    add_formation(blue, Infantry, count=15, px=110, py=15, sx=110, sy=45)
    world.add_team(red)
    world.add_team(blue)
    def step(t):
```

```
    if t == 20:
        cav_form.move(px=100, py=25, sx=100, py=35)
run(root, world, view, step_hook=step, duration_ticks=1200)
```

Snippet 37: TestTrainingEnvironment

```
from gym_wrapper import BattleEnv, OBS_SIZE, MAX_FORMATIONS, ACTIONS_PER_FORMATION


class TestTrainingEnvironment:

    def test_observation_space_shape(self):
        env = BattleEnv(curriculum_stage=0)
        obs, _ = env.reset()
        assert obs.shape == (OBS_SIZE,)

    def test_observation_values_in_range(self):
        env = BattleEnv(curriculum_stage=0)
        obs, _ = env.reset()
        assert np.all(obs >= 0.0)
        assert np.all(obs <= 1.0)

    def test_action_space_shape(self):
        env = BattleEnv(curriculum_stage=0)
        expected = (MAX_FORMATIONS * ACTIONS_PER_FORMATION,)
        assert env.action_space.shape == expected

    def test_step_returns_correct_tuple(self):
        env = BattleEnv(curriculum_stage=0)
        env.reset()
        action = env.action_space.sample()
        result = env.step(action)
        assert len(result) == 5
        obs, reward, terminated, truncated, info = result
        assert obs.shape == (OBS_SIZE,)
        assert isinstance(reward, float)
        assert isinstance(terminated, bool)
        assert isinstance(truncated, bool)
        assert isinstance(info, dict)
```

```
  def test_reset_produces_different_maps(self):
    env = BattleEnv(curriculum_stage=0)
    obs1, _ = env.reset(seed=1)
    obs2, _ = env.reset(seed=2)
    assert not np.array_equal(obs1, obs2)

  def test_dead_formations_zeroed_in_obs(self):
    env = BattleEnv(curriculum_stage=0)
    env.reset()
    first_formation = env.friendly_team.get_formations()[0]
    for u in first_formation.get_units():
      u.set_hp(0)
    obs = env._build_obs()
    slot = obs[0:14]
    assert np.all(slot == 0.0)
```

Snippet 38: TestSelfPlayMirroring

```
class TestSelfPlayMirroring:

  def test_mirrored_obs_same_shape(self):
    env = BattleEnv(curriculum_stage=0)
    env.reset()
    assert env._build_obs().shape == env._build_opponent_obs().shape

  def test_mirrored_obs_swaps_teams(self):
    env = BattleEnv(curriculum_stage=0)
    env.reset()
    friendly_obs = env._build_obs()
    opponent_obs = env._build_opponent_obs()
    friendly_slot_size = MAX_FORMATIONS * 14
    friendly_presence = friendly_obs[0]
    opponent_enemy_presence = opponent_obs[friendly_slot_size]
    if friendly_presence == 1.0:
      assert opponent_enemy_presence == 1.0
```

```
  def test_mirrored_x_coordinates_sum_to_one(self):
    env = BattleEnv(curriculum_stage=0)
    env.reset()
    friendly_obs = env._build_obs()
    opponent_obs = env._build_opponent_obs()
    friendly_slot_size = MAX_FORMATIONS * 14
    enemy_avg_x_normal = friendly_obs[friendly_slot_size + 11]
    enemy_avg_x_mirrored = opponent_obs[0 + 11]
    if enemy_avg_x_normal > 0:
      total = enemy_avg_x_normal + enemy_avg_x_mirrored
      assert abs(total - 1.0) < 0.05


  def test_global_features_unchanged_by_mirror(self):
    env = BattleEnv(curriculum_stage=0)
    env.reset()
    friendly_obs = env._build_obs()
    opponent_obs = env._build_opponent_obs()
    global_offset = MAX_FORMATIONS * 14 * 2
    assert np.allclose(
      friendly_obs[global_offset:],
      opponent_obs[global_offset:]
    )
```

Snippet 39: TestRewardValidation

```
class TestRewardValidation:


  def test_reward_is_finite(self):
    env = BattleEnv(curriculum_stage=0)
    env.reset()
    for _ in range(50):
      action = env.action_space.sample()
      _, reward, terminated, truncated, _ = env.step(action)
      assert np.isfinite(reward)
      if terminated or truncated:
        break
```

```
def test_victory_gives_positive_terminal_reward(self):
    env = BattleEnv(curriculum_stage=0)
    env.reset()
    for f in env.enemy_team.get_formations():
        for u in f.get_units():
            u.set_hp(0)
    action = np.zeros(MAX_FORMATIONS * ACTIONS_PER_FORMATION, dtype=np.float32)
    _, reward, terminated, _, info = env.step(action)
    assert terminated
    assert info.get('won') == True
    assert reward > 0


def test_defeat_gives_negative_terminal_reward(self):
    env = BattleEnv(curriculum_stage=0)
    env.reset()
    for f in env.friendly_team.get_formations():
        for u in f.get_units():
            u.set_hp(0)
    action = env.action_space.sample()
    _, reward, terminated, _, info = env.step(action)
    assert terminated
    assert info.get('won') == False
    assert reward < 0


def test_shield_wall_on_cavalry_penalised(self):
    env = BattleEnv(curriculum_stage=0)
    env.reset()
    cav_index = None
    for i, f in enumerate(env.friendly_team.get_formations()):
        living = f.get_living_units()
        if living and isinstance(living[0], model.Cavalry):
            cav_index = i
            break
    if cav_index is not None:
        action = np.zeros(MAX_FORMATIONS * ACTIONS_PER_FORMATION, dtype=np.float32)
        action[cav_index * ACTIONS_PER_FORMATION + 4] = 1.0
        _, reward, _, _, _ = env.step(action)
        assert reward < -5.0
```

```
  def test_episode_truncates_at_max_steps(self):
      env = BattleEnv(curriculum_stage=0)
      env.reset()
      env.current_step = 1999
      action = env.action_space.sample()
      _, _, _, truncated, _ = env.step(action)
      assert truncated
```

Snippet 40: TestPerformance

```
import time


class TestPerformance:


  def test_tick_performance_small(self):
      world = model.World(300, 300, tick_rate=10, curriculum_stage=3)
      team_a = model.Team(team_id=False)
      team_b = model.Team(team_id=True)
      np.random.seed(0)
      world.populate_team(team_a)
      world.populate_team(team_b)
      for _ in range(5):
          world.tick()
      start = time.perf_counter()
      for _ in range(100):
          world.tick()
      elapsed = (time.perf_counter() - start) / 100
      assert elapsed < 0.01


  def test_tick_performance_large(self):
      world = model.World(300, 300, tick_rate=10, curriculum_stage=3)
      team_a = model.Team(team_id=False)
      team_b = model.Team(team_id=True)
      for _ in range(4):
          units = [model.Infantry() for _ in range(50)]
          for u in units:
```

```
            u.set_team(False)
        f = model.Formation(units)
        f.move(50, 50, 150, 250)
        team_a.add_formation(f)
    for _ in range(4):
        units = [model.Infantry() for _ in range(50)]
        for u in units:
            u.set_team(True)
        f = model.Formation(units)
        f.move(150, 50, 250, 250)
        team_b.add_formation(f)
    world.add_team(team_a)
    world.add_team(team_b)
    for _ in range(5):
        world.tick()
    start = time.perf_counter()
    for _ in range(50):
        world.tick()
    elapsed = (time.perf_counter() - start) / 50
    assert elapsed < 0.05


def test_env_reset_performance(self):
    env = BattleEnv(curriculum_stage=3)
    start = time.perf_counter()
    for _ in range(10):
        env.reset()
    elapsed = (time.perf_counter() - start) / 10
    assert elapsed < 0.5
```

Snippet 41: TestCurriculumStateMachine

python

```
class TestCurriculumStateMachine:
    def _make_callback(self, threshold=0.7, window=10, max_stage=3):
        cb = CurriculumSelfPlayCallback(
            save_path=tempfile.mkdtemp(),
            promotion_threshold=threshold,
            window_size=window,
            max_stage=max_stage,
```

```
        verbose=0,
    )
    cb.model = MagicMock()
    mock_env = MagicMock()
    cb.model.get_env.return_value = mock_env
    cb.locals = {"dones": [], "infos": []}
    return cb

def _feed_outcomes(self, cb, wins, losses):
    for _ in range(wins):
        cb.locals = {"dones": [True], "infos": [{"won": True}]}
        cb._on_step()
    for _ in range(losses):
        cb.locals = {"dones": [True], "infos": [{"won": False}]}
        cb._on_step()

def test_starts_at_stage0_vs_scripted(self):
    cb = self._make_callback()
    assert cb.current_stage == 0
    assert cb.vs_self_play == False
    assert cb.fully_complete == False

def test_no_promotion_below_threshold(self):
    cb = self._make_callback(threshold=0.7, window=10)
    self._feed_outcomes(cb, wins=6, losses=4)
    assert cb.current_stage == 0
    assert cb.vs_self_play == False

def test_promotes_to_self_play_at_threshold(self):
    cb = self._make_callback(threshold=0.7, window=10)
    self._feed_outcomes(cb, wins=8, losses=2)
    assert cb.vs_self_play == True
    assert cb.current_stage == 0
    cb.model.save.assert_called_once()
    cb.training_env.env_method.assert_any_call(
        "set_opponent_model", cb.model.save.call_args[0][0])
```

```
def test_promotes_to_next_stage_after_self_play(self):
    cb = self._make_callback(threshold=0.7, window=10)
    self._feed_outcomes(cb, wins=8, losses=2)
    self._feed_outcomes(cb, wins=8, losses=2)
    assert cb.vs_self_play == False
    assert cb.current_stage == 1
    cb.training_env.env_method.assert_any_call("set_opponent_model", None)
    cb.training_env.env_method.assert_any_call("set_curriculum_stage", 1)


def test_full_curriculum_completion(self):
    cb = self._make_callback(threshold=0.7, window=10, max_stage=3)
    for expected_stage in range(4):
        assert cb.current_stage == expected_stage
        assert cb.vs_self_play == False
        self._feed_outcomes(cb, wins=8, losses=2)
        assert cb.vs_self_play == True
        self._feed_outcomes(cb, wins=8, losses=2)
    assert cb.fully_complete == True


def test_no_further_transitions_after_complete(self):
    cb = self._make_callback(threshold=0.7, window=10, max_stage=0)
    self._feed_outcomes(cb, wins=8, losses=2)
    self._feed_outcomes(cb, wins=8, losses=2)
    assert cb.fully_complete == True
    call_count = cb.model.save.call_count
    self._feed_outcomes(cb, wins=10, losses=0)
    assert cb.model.save.call_count == call_count


def test_episode_outcomes_cleared_on_promotion(self):
    cb = self._make_callback(threshold=0.7, window=10)
    self._feed_outcomes(cb, wins=8, losses=2)
    assert len(cb.episode_outcomes) == 0


def test_window_uses_most_recent(self):
    cb = self._make_callback(threshold=0.7, window=10)
    self._feed_outcomes(cb, wins=0, losses=5)
    self._feed_outcomes(cb, wins=10, losses=0)
    assert cb.vs_self_play == True
```

Snippet 42: TestModelLoading

python

```
@pytest.mark.skipif(not HAS_SB3, reason="sb3_contrib not installed")
class TestModelLoading:
    CHECKPOINT = "battle_agent_finetuned.zip"


    def test_model_loads_and_predicts(self):
        if not os.path.exists(self.CHECKPOINT):
            pytest.skip("No checkpoint available")
        loaded_model = RecurrentPPO.load(self.CHECKPOINT)
        env = BattleEnv(curriculum_stage=3)
        obs, _ = env.reset()
        lstm_states = None
        episode_start = np.ones((1,), dtype=bool)
        obs_batch = np.expand_dims(obs, axis=0)
        action, lstm_states = loaded_model.predict(
            obs_batch, state=lstm_states,
            episode_start=episode_start, deterministic=True)
        action = action.squeeze(0)
        assert action.shape == (MAX_FORMATIONS * ACTIONS_PER_FORMATION,)
        assert np.all(np.isfinite(action))


    def test_opponent_snapshot_loads_into_env(self):
        if not os.path.exists(self.CHECKPOINT):
            pytest.skip("No checkpoint available")
        env = BattleEnv(curriculum_stage=3)
        env.reset()
        env.set_opponent_model(self.CHECKPOINT)
        for _ in range(10):
            action = np.zeros(MAX_FORMATIONS * ACTIONS_PER_FORMATION,
                              dtype=np.float32)
            obs, reward, terminated, truncated, info = env.step(action)
            assert np.all(np.isfinite(obs))
            if terminated or truncated:
                break


    def test_empty_opponent_snapshots_handled(self):
```

```
    env = BattleEnv(curriculum_stage=3)
    env.reset()
    env.set_opponent_model(None)
    action = np.zeros(MAX_FORMATIONS * ACTIONS_PER_FORMATION,
                      dtype=np.float32)
    obs, reward, terminated, truncated, info = env.step(action)
    assert np.all(np.isfinite(obs))
    assert np.isfinite(reward)
```

Snippet 43: TestEarlyVsLateModel

python

```
@pytest.mark.skipif(not HAS_SB3, reason="sb3_contrib not installed")
class TestEarlyVsLateModel:
    EARLY_CHECKPOINT = "battle_agent_finetuned.zip"
    LATE_CHECKPOINT = "battle_agent_ultra_finetuned.zip"
    NUM_EVAL_EPISODES = 5
    MAX_EVAL_STEPS = 500


    def _evaluate_model(self, model_path, num_episodes):
        loaded_model = RecurrentPPO.load(model_path)
        env = BattleEnv(curriculum_stage=3, frame_skip=5)
        wins = 0
        total_reward = 0.0
        for _ in range(num_episodes):
            obs, _ = env.reset()
            lstm_states = None
            episode_start = np.ones((1,), dtype=bool)
            episode_reward = 0.0
            steps = 0
            while steps < self.MAX_EVAL_STEPS:
                obs_batch = np.expand_dims(obs, axis=0)
                action, lstm_states = loaded_model.predict(
                    obs_batch, state=lstm_states,
                    episode_start=episode_start, deterministic=True)
                episode_start = np.zeros((1,), dtype=bool)
                action = action.squeeze(0)
                obs, reward, terminated, truncated, info = env.step(action)
                episode_reward += reward
                steps += 1
```

```
            if terminated or truncated:
                break
        if info.get("won"):
            wins += 1
        total_reward += episode_reward
    return wins / num_episodes, total_reward / num_episodes


def test_late_model_wins_more_than_early(self):
    if not os.path.exists(self.EARLY_CHECKPOINT):
        pytest.skip(f"Not found: {self.EARLY_CHECKPOINT}")
    if not os.path.exists(self.LATE_CHECKPOINT):
        pytest.skip(f"Not found: {self.LATE_CHECKPOINT}")
    early_wr, early_reward = self._evaluate_model(
        self.EARLY_CHECKPOINT, self.NUM_EVAL_EPISODES)
    late_wr, late_reward = self._evaluate_model(
        self.LATE_CHECKPOINT, self.NUM_EVAL_EPISODES)
    print(f"\nEarly: wr={early_wr:.0%}, reward={early_reward:.1f}")
    print(f"Late:  wr={late_wr:.0%}, reward={late_reward:.1f}")
    assert late_wr >= early_wr


def test_late_model_higher_avg_reward(self):
    if not os.path.exists(self.EARLY_CHECKPOINT):
        pytest.skip(f"Not found: {self.EARLY_CHECKPOINT}")
    if not os.path.exists(self.LATE_CHECKPOINT):
        pytest.skip(f"Not found: {self.LATE_CHECKPOINT}")
    _, early_reward = self._evaluate_model(
        self.EARLY_CHECKPOINT, self.NUM_EVAL_EPISODES)
    _, late_reward = self._evaluate_model(
        self.LATE_CHECKPOINT, self.NUM_EVAL_EPISODES)
    assert late_reward > early_reward
```

Appendix - NFT Code Snippets

Snippet 44: NFT-1: Memory Stability (Section 7.3.1)

```
class TestMemoryStability(unittest.TestCase):
    WARMUP_STEPS   = 300
    SAMPLE_STEPS   = 1000
    SAMPLE_INTERVAL = 200
```

```
    def test_memory_stabilises_after_warmup(self):
        env = BattleEnv(curriculum_stage=3)
        env.reset(seed=0)
        tracemalloc.start()
        self._run_env_steps(env, self.WARMUP_STEPS, seed_offset=1000)
        samples = []
        def _record(step):
            if step % self.SAMPLE_INTERVAL == 0:
                current, _ = tracemalloc.get_traced_memory()
                samples.append(current)
        self._run_env_steps(env, self.SAMPLE_STEPS, record_fn=_record)
        tracemalloc.stop()
        growth_pct = (samples[-1] - samples[0]) / max(samples[0], 1) * 100
        assert growth_pct < 20.0

    def test_current_memory_under_10mb_per_episode(self):
        env = BattleEnv(curriculum_stage=3)
        env.reset(seed=1)
        tracemalloc.start()
        for step in range(500):
            _, _, terminated, truncated, _ = env.step(env.action_space.sample())
            if terminated or truncated:
                env.reset(seed=step)
                break
        gc.collect()
        current, _ = tracemalloc.get_traced_memory()
        tracemalloc.stop()
        assert current < 10 * 1024 * 1024
```

Snippet 45: NFT-2: Simulation Determinism (Section 7.3.2)

```
class TestDeterminismUnderConcurrency(unittest.TestCase):

    def test_sequential_same_seed_fully_deterministic(self):
        def run(seed):
            np.random.seed(seed)
            env = BattleEnv(curriculum_stage=0)
            obs, _ = env.reset(seed=seed)
```

```
        rng = np.random.default_rng(seed)
        trace = [obs.copy()]
        for _ in range(50):
            action = rng.uniform(-1, 1, size=env.action_space.shape).astype(np.float32)
            obs, _, terminated, truncated, _ = env.step(action)
            trace.append(obs.copy())
            if terminated or truncated:
                break
        return trace
    trace_1 = run(7)
    trace_2 = run(7)
    for o1, o2 in zip(trace_1, trace_2):
        assert np.allclose(o1, o2, atol=1e-6)

def test_parallel_envs_no_crash_no_nan(self):
    errors = []
    def _run(seed, steps, key):
        env = BattleEnv(curriculum_stage=3)
        env.reset(seed=seed)
        rng = np.random.default_rng(seed + 1000)
        for _ in range(steps):
            obs, reward, terminated, truncated, _ = env.step(
                rng.uniform(-1, 1, size=env.action_space.shape).astype(np.float32))
            if not np.all(np.isfinite(obs)):
                errors.append(f"[{key}] NaN/Inf in observation")
                break
            if terminated or truncated:
                env.reset(seed=seed)
    t1 = threading.Thread(target=_run, args=(42, 200, "thread-A"))
    t2 = threading.Thread(target=_run, args=(99, 200, "thread-B"))
    t1.start(); t2.start()
    t1.join();  t2.join()
    assert not errors
```

Snippet 46: NFT-3: Graceful Degradation (Section 7.3.3)

```
class TestGracefulDegradation(unittest.TestCase):
```

```
def _make_large_world(self, units_per_side):
    world = model.World(300, 300, tick_rate=10, curriculum_stage=3)
    for team_id in [False, True]:
        team = model.Team(team_id=team_id)
        units = [model.Infantry() for _ in range(units_per_side)]
        for u in units: u.set_team(team_id)
        formation = model.Formation(units)
        if not team_id: formation.move(10, 10, 10, 290)
        else:           formation.move(290, 10, 290, 290)
        for u in formation.get_units():
            dest = u.get_destination()
            if dest: u.set_x(dest[0]); u.set_y(dest[1])
        team.add_formation(formation)
        world.add_team(team)
    return world

def test_500_units_per_side_no_crash_no_nan(self):
    world = self._make_large_world(500)
    for _ in range(50): world.tick()
    for unit in world.get_all_units():
        assert not math.isnan(unit.get_x())
        assert not math.isnan(unit.get_y())

def test_500_units_per_side_tick_time_recorded(self):
    world = self._make_large_world(500)
    times = []
    for _ in range(100):
        t0 = time.perf_counter()
        world.tick()
        times.append(time.perf_counter() - t0)
    avg_ms = (sum(times) / len(times)) * 1000
    assert avg_ms < 500

def test_extreme_unit_count_units_stay_in_bounds(self):
    world = self._make_large_world(500)
    for _ in range(50): world.tick()
    for unit in world.get_all_units():
        assert 0 <= unit.get_x() <= world.width
```

```
        assert 0 <= unit.get_y() <= world.height
```

Snippet 47: NFT-4: Visualiser Frame Rate (Section 7.3.4)

```
# WorldView.render() is monkey-patched to record per-call duration.
# root.after() keeps the tkinter event loop alive between frames
# so the Canvas actually flushes draw commands to the display.

render_durations_ms = []
_original_render = view.render

def _timed_render():
    t0 = time.perf_counter()
    _original_render()
    render_durations_ms.append((time.perf_counter() - t0) * 1000)

view.render = _timed_render

frames_done = [0]
def _step():
    if frames_done[0] >= FRAME_COUNT:
        root.quit()
        return
    controller._do_tick()
    frames_done[0] += 1
    root.after(MS_PER_FRAME, _step)

root.after(MS_PER_FRAME, _step)
root.mainloop()

avg_ms  = sum(render_durations_ms) / len(render_durations_ms)
peak_ms = max(render_durations_ms)
assert avg_ms  <= MAX_AVG_MS    # threshold: 100 ms
assert peak_ms <= MAX_PEAK_MS   # threshold: 300 ms
```

Snippet 48: NFT-5: Observation Numerical Stability (Section 7.3.5)

```
class TestObservationNumericalStability(unittest.TestCase):
```

```
def test_no_nan_in_obs_over_full_episode(self):
    env = BattleEnv(curriculum_stage=3)
    obs, _ = env.reset(seed=99)
    assert np.all(np.isfinite(obs))
    for step in range(2000):
        obs, reward, terminated, truncated, _ = env.step(env.action_space.sample())
        assert np.all(np.isfinite(obs)), f"NaN/Inf at step {step}"
        assert math.isfinite(reward),    f"Non-finite reward at step {step}"
        if terminated or truncated:
            obs, _ = env.reset(seed=step)
            break

def test_obs_within_declared_bounds(self):
    env = BattleEnv(curriculum_stage=3)
    obs, _ = env.reset(seed=5)
    for step in range(500):
        obs, _, terminated, truncated, _ = env.step(env.action_space.sample())
        assert np.all(obs >= 0.0), f"Obs below 0 at step {step}"
        assert np.all(obs <= 1.0), f"Obs above 1 at step {step}"
        if terminated or truncated:
            obs, _ = env.reset(seed=step)

def test_reward_remains_finite_over_500_random_steps(self):
    for stage in range(4):
        env = BattleEnv(curriculum_stage=stage)
        env.reset(seed=stage * 10)
        for step in range(500):
            _, reward, terminated, truncated, _ = env.step(env.action_space.sample())
            assert math.isfinite(reward)
            if terminated or truncated:
                env.reset(seed=step)
                break
```

Appendix - AT Code Snippets

Snippet 49: AT-1: Agent Behaviour Plausibility (Section 7.4.1)

```
# Scoring rubric (1-5):
#   1 - Completely random, no visible intent
```

```
#   2 - Some movement toward enemy but no tactics
#   3 - Engages enemy and occasionally reacts to threats
#   4 - Uses formations deliberately, shield wall used defensively
#   5 - Coordinated multi-formation behaviour, clear tactical logic
# Pass criterion: mean score >= 3.0

scores = []
for episode in range(1, 6):
    input("Press Enter to begin scoring episode... ")
    while True:
        raw = input(f"Score for episode {episode} (1-5): ").strip()
        score = int(raw)
        if 1 <= score <= 5:
            scores.append(score)
            break

mean_score = sum(scores) / len(scores)
passed = mean_score >= 3.0
results["AT-1"] = {"scores": scores, "mean": mean_score, "passed": passed}

# Results: scores [4, 5, 3, 4, 4], mean = 4.0 — PASS
```

Snippet 50: AT-2: Curriculum Progression Verifiability (Section 7.4.2)

```
class TestCurriculumProgression(unittest.TestCase):

    def test_tensorboard_log_directory_present(self):
        assert os.path.isdir("tb_logs")

    def test_tensorboard_readable_and_contains_scalars(self):
        ea = self._load_accumulator()
        tags = ea.Tags().get("scalars", [])
        assert len(tags) > 0

    def test_episode_reward_present_in_logs(self):
        # SB3 logs rollout/ep_rew_mean only when a full episode
        # completes within an n_steps rollout window. With
        # SubprocVecEnv and long episodes this tag was absent.
```

```
    ea = self._load_accumulator()
    tags = ea.Tags().get("scalars", [])
    assert "rollout/ep_rew_mean" in tags  # FAILED — see Section 7.4.2

# Available tags confirmed present:
# time/fps, train/approx_kl, train/clip_fraction, train/clip_range,
# train/entropy_loss, train/explained_variance, train/learning_rate,
# train/loss, train/policy_gradient_loss, train/std, train/value_loss
# rollout/ep_rew_mean: ABSENT — episodes exceeded n_steps horizon
# without terminating, so SB3 never logged a completed episode reward.
```

Snippet 51: AT-3: Visualiser Interpretability (Section 7.4.3)

```
# Questions posed to an evaluator with no prior knowledge of the system:
#   Q1: Which side appears to be winning?
#   Q2: Can you identify any distinct formation or group shape?
#   Q3: Did you notice any special behaviour or mechanic?
#
# Recorded responses:
#   Q1: "right"
#   Q2: "sometimes they are like my siege team mates where its one on one and lose"
#   Q3: "their not very smart"
#
# Pass criterion: all three answered with a non-empty, confident response.

def _answered(text):
   return bool(text) and text.lower() not in {"unclear", "no", "none", "n/a"}

passed = all(_answered(a) for a in answers.values())
# Result: PASS
# Q3 additionally provides qualitative evidence that the agent's
# behaviour was perceived as unintelligent by an independent observer.
```

Snippet 52: AT-4: Dissertation Reproducibility (Section 7.4.4)

```
class TestReproducibility(unittest.TestCase):

   def test_required_dependencies_installed(self):
```

```
    required = ["gymnasium", "numpy", "scipy", "stable_baselines3"]
    missing = []
    for pkg in required:
      try:
        importlib.import_module(pkg)
      except ImportError:
        missing.append(pkg)
    assert not missing

  def test_env_resets_without_error(self):
    env = BattleEnv(curriculum_stage=0)
    obs, info = env.reset(seed=0)
    assert obs is not None

  def test_checkpoint_present_or_training_can_start(self):
    checkpoint_paths = ["battle_agent_finetuned.zip",
                "battle_agent_ultra_finetuned.zip"]
    checkpoint_found = any(Path(p).exists() for p in checkpoint_paths)
    if not checkpoint_found:
      vec_env = DummyVecEnv([lambda: BattleEnv(curriculum_stage=0)])
      new_model = RecurrentPPO("MlpLstmPolicy", vec_env, verbose=0)
      assert new_model is not None
```

Appendix – Test Results Snippets

Test output 1: test.py (99 tests)

```
=== test session starts ===
platform win32 -- Python 3.11.9, pytest-9.0.3
collected 99 items

test.py::TestUnitStats::test_infantry_stats PASSED                [  1%]
test.py::TestUnitStats::test_archer_stats PASSED                  [  2%]
test.py::TestUnitStats::test_cavalry_stats PASSED                 [  3%]
test.py::TestUnitDeath::test_unit_dies_at_zero_hp PASSED          [  4%]
test.py::TestUnitDeath::test_unit_dies_below_zero_hp PASSED       [  5%]
test.py::TestUnitDeath::test_unit_alive_at_one_hp PASSED          [  6%]
test.py::TestUnitMovement::test_unit_moves_toward_destination PASSED  [  7%]
test.py::TestUnitMovement::test_unit_does_not_overshoot PASSED    [  8%]
```

test.py::TestUnitMovement::test_cavalry_faster_than_infantry PASSED [9%]

test.py::TestUnitMovement::test_attacking_unit_stops_moving PASSED [10%]

test.py::TestUnitMovement::test_cavalry_moves_while_attacking PASSED [11%]

test.py::TestShieldWall::test_shield_wall_halves_outgoing_damage PASSED [12%]

test.py::TestShieldWall::test_no_wall_full_damage PASSED [13%]

test.py::TestShieldWall::test_archer_vs_shield_wall_95_percent_reduction PASSED [14%]

test.py::TestShieldWall::test_archer_vs_no_wall_full_damage PASSED [15%]

test.py::TestShieldWall::test_archer_vs_non_infantry_full_damage PASSED [16%]

test.py::TestShieldWall::test_shield_wall_halves_movement_speed PASSED [17%]

test.py::TestFormation::test_units_receive_destinations PASSED [18%]

test.py::TestFormation::test_destinations_within_bounds PASSED [19%]

test.py::TestFormation::test_rank_count_increases_with_depth PASSED [20%]

test.py::TestFormation::test_single_unit_formation PASSED [21%]

test.py::TestFormation::test_empty_formation_no_crash PASSED [22%]

test.py::TestFormation::test_living_units_excludes_dead PASSED [23%]

test.py::TestTeam::test_team_defeated_when_all_dead PASSED [24%]

test.py::TestTeam::test_team_not_defeated_with_living PASSED [25%]

test.py::TestTeam::test_get_living_units_across_formations PASSED [26%]

test.py::TestTerrainGeneration::test_terrain_map_shape PASSED [27%]

test.py::TestTerrainGeneration::test_height_map_shape PASSED [28%]

test.py::TestTerrainGeneration::test_terrain_values_valid PASSED [29%]

test.py::TestTerrainGeneration::test_height_values_in_range PASSED [30%]

test.py::TestTerrainGeneration::test_water_tiles_height_zero PASSED [31%]

test.py::TestTerrainGeneration::test_terrain_deterministic_with_seed PASSED [32%]

test.py::TestSpeedModifiers::test_stage0_always_returns_1 PASSED [33%]

test.py::TestSpeedModifiers::test_stage1_water_slows PASSED [34%]

test.py::TestSpeedModifiers::test_stage1_plains_normal PASSED [35%]

test.py::TestConcealment::test_stage0_no_concealment PASSED [36%]

test.py::TestConcealment::test_stage2_forest_conceals PASSED [37%]

test.py::TestConcealment::test_concealed_unit_detected_within_range PASSED [38%]

test.py::TestConcealment::test_concealed_unit_not_detected_outside_range PASSED [39%]

test.py::TestLineOfSight::test_los_always_true_below_stage3 PASSED [40%]

test.py::TestLineOfSight::test_los_blocked_by_peak PASSED [41%]

test.py::TestLineOfSight::test_los_not_blocked_by_equal_height PASSED [42%]

test.py::TestLineOfSight::test_los_high_ground_sees_over_low_peak PASSED [43%]

test.py::TestHeightRangeBonus::test_higher_ground_increases_range PASSED [44%]

test.py::TestHeightRangeBonus::test_lower_ground_decreases_range PASSED [45%]

test.py::TestHeightRangeBonus::test_no_range_bonus_before_stage3 PASSED [46%]

```
test.py::TestCombat::test_units_in_range_deal_damage PASSED          [ 47%]
test.py::TestCombat::test_units_out_of_range_no_damage PASSED        [ 48%]
test.py::TestCombat::test_dead_units_do_not_attack PASSED            [ 49%]
test.py::TestCollisionResolution::test_overlapping_units_separated PASSED [ 50%]
test.py::TestCollisionResolution::test_non_overlapping_units_unchanged PASSED [ 51%]
test.py::TestCollisionResolution::test_units_clamped_to_world_bounds PASSED [ 52%]
test.py::TestCohesion::test_cohesion_in_valid_range PASSED           [ 53%]
test.py::TestCohesion::test_perfect_line_high_cohesion PASSED        [ 54%]
test.py::TestCohesion::test_single_unit_cohesion_is_one PASSED       [ 55%]
test.py::TestCohesion::test_scattered_units_low_cohesion PASSED      [ 56%]
test.py::TestPopulation::test_populated_team_has_formations PASSED   [ 57%]
test.py::TestPopulation::test_populated_team_has_living_units PASSED [ 58%]
test.py::TestPopulation::test_all_units_have_correct_team_id PASSED  [ 59%]
test.py::TestPopulation::test_units_within_world_bounds PASSED       [ 60%]
test.py::TestPopulation::test_max_six_formations PASSED              [ 61%]
test.py::TestTick::test_tick_increments_count PASSED                 [ 62%]
test.py::TestTick::test_100_ticks_no_crash PASSED                    [ 63%]
test.py::TestTick::test_battle_eventually_ends PASSED                [ 64%]
test.py::TestBresenham::test_horizontal_line PASSED                  [ 65%]
test.py::TestBresenham::test_vertical_line PASSED                    [ 66%]
test.py::TestBresenham::test_single_point PASSED                     [ 67%]
test.py::TestBresenham::test_includes_start_and_end PASSED           [ 68%]
test.py::TestTrainingEnvironment::test_observation_space_shape PASSED  [ 69%]
test.py::TestTrainingEnvironment::test_observation_values_in_range PASSED [ 70%]
test.py::TestTrainingEnvironment::test_action_space_shape PASSED     [ 71%]
test.py::TestTrainingEnvironment::test_step_returns_correct_tuple PASSED [ 72%]
test.py::TestTrainingEnvironment::test_reset_produces_different_maps PASSED [ 73%]
test.py::TestTrainingEnvironment::test_dead_formations_zeroed_in_obs PASSED [ 74%]
test.py::TestSelfPlayMirroring::test_mirrored_obs_same_shape PASSED  [ 75%]
test.py::TestSelfPlayMirroring::test_mirrored_obs_swaps_teams PASSED [ 76%]
test.py::TestSelfPlayMirroring::test_mirrored_x_coordinates_sum_to_one PASSED [ 77%]
test.py::TestSelfPlayMirroring::test_global_features_unchanged_by_mirror PASSED [ 78%]
test.py::TestRewardValidation::test_reward_is_finite PASSED          [ 79%]
test.py::TestRewardValidation::test_victory_gives_positive_terminal_reward PASSED [ 80%]
test.py::TestRewardValidation::test_defeat_gives_negative_terminal_reward PASSED [ 81%]
test.py::TestRewardValidation::test_shield_wall_on_cavalry_penalised PASSED [ 82%]
test.py::TestRewardValidation::test_episode_truncates_at_max_steps PASSED [ 83%]
test.py::TestPerformance::test_tick_performance_small PASSED         [ 84%]
```

test.py::TestPerformance::test_tick_performance_large PASSED [85%]

test.py::TestPerformance::test_env_reset_performance PASSED [86%]

test.py::TestCurriculumStateMachine::test_starts_at_stage0_vs_scripted PASSED [87%]

test.py::TestCurriculumStateMachine::test_no_promotion_below_threshold PASSED [88%]

test.py::TestCurriculumStateMachine::test_promotes_to_self_play_at_threshold PASSED [89%]

test.py::TestCurriculumStateMachine::test_promotes_to_next_stage_after_self_play PASSED [90%]

test.py::TestCurriculumStateMachine::test_full_curriculum_completion PASSED [91%]

test.py::TestCurriculumStateMachine::test_no_further_transitions_after_complete PASSED [92%]

test.py::TestCurriculumStateMachine::test_episode_outcomes_cleared_on_promotion PASSED [93%]

test.py::TestCurriculumStateMachine::test_window_uses_most_recent PASSED [94%]

test.py::TestModelLoading::test_model_loads_and_predicts PASSED [95%]

test.py::TestModelLoading::test_opponent_snapshot_loads_into_env PASSED [96%]

test.py::TestModelLoading::test_empty_opponent_snapshots_handled PASSED [97%]

test.py::TestEarlyVsLateModel::test_late_model_wins_more_than_early PASSED [98%]

test.py::TestEarlyVsLateModel::test_late_model_higher_avg_reward PASSED [100%]

99 passed in 295.86s (0:04:55)

Test output 2: test_a.py (14 tests)

=== test session starts ===

collected 14 items

test_a.py::TestReproducibility::test_model_module_importable PASSED [7%]

test_a.py::TestReproducibility::test_gym_wrapper_importable PASSED [14%]

test_a.py::TestReproducibility::test_required_dependencies_installed PASSED [21%]

test_a.py::TestReproducibility::test_sb3_contrib_installed PASSED [28%]

test_a.py::TestReproducibility::test_env_resets_without_error PASSED [35%]

test_a.py::TestReproducibility::test_env_steps_without_error PASSED [42%]

test_a.py::TestReproducibility::test_checkpoint_present_or_training_can_start PASSED [50%]

test_a.py::TestCurriculumProgression::test_tensorboard_log_directory_present PASSED [57%]

test_a.py::TestCurriculumProgression::test_tensorboard_readable_and_contains_scalars PASSED [64%]

test_a.py::TestCurriculumProgression::test_episode_reward_present_in_logs FAILED [71%]

test_a.py::TestCurriculumProgression::test_reward_trend_is_non_catastrophic SKIPPED [78%]

test_a.py::TestCurriculumProgression::test_training_ran_for_expected_duration SKIPPED [85%]

test_a.py::TestHumanEvaluation::test_at1_agent_plausibility_results_on_disk PASSED [92%]

test_a.py::TestHumanEvaluation::test_at3_visualiser_interpretability_results_on_disk PASSED [100%]

FAILED: test_episode_reward_present_in_logs

rollout/ep_rew_mean not found. Available: time/fps, train/approx_kl,

train/clip_fraction, train/clip_range, train/entropy_loss,

train/explained_variance, train/learning_rate, train/loss,

train/policy_gradient_loss, train/std, train/value_loss

1 failed, 11 passed, 2 skipped in 65.48s

Test output 3: test_nf.py (10 tests)

=== test session starts ===

collected 10 items

test_nf.py::TestMemoryStability::test_memory_stabilises_after_warmup FAILED [10%]

test_nf.py::TestMemoryStability::test_current_memory_under_10mb_per_episode PASSED [20%]

test_nf.py::TestDeterminismUnderConcurrency::test_sequential_same_seed_fully_deterministic PASSED [30%]

test_nf.py::TestDeterminismUnderConcurrency::test_parallel_envs_no_crash_no_nan PASSED [40%]

test_nf.py::TestGracefulDegradation::test_500_units_per_side_no_crash_no_nan PASSED [50%]

test_nf.py::TestGracefulDegradation::test_500_units_per_side_tick_time_recorded PASSED [60%]

test_nf.py::TestGracefulDegradation::test_extreme_unit_count_units_stay_in_bounds PASSED [70%]

test_nf.py::TestObservationNumericalStability::test_no_nan_in_obs_over_full_episode PASSED [80%]

test_nf.py::TestObservationNumericalStability::test_obs_within_declared_bounds PASSED [90%]

test_nf.py::TestObservationNumericalStability::test_reward_remains_finite_over_500_random_steps PASSED [100%]

FAILED: test_memory_stabilises_after_warmup

Current memory grew 63.3% after warmup.

Samples: [70483, 86061, 95421, 105281, 115085]

assert 63.3 < 20.0

1 failed, 9 passed in 208.77s

Test output 4: test_nft4_vis.py (frame rate)

[NFT-4] Measuring 200 frames (200v200 infantry) ...

[NFT-4] Results — 201 frames, 200v200 units

Average : 5.24 ms (threshold <= 100.0 ms)

95th pct : 6.00 ms

Peak : 13.29 ms (threshold <= 300.0 ms)

Min : 4.66 ms

~FPS : 190.8 avg

[NFT-4] PASS

Test output 5: AT-1 and AT-3 results (JSON)


```
{
 "AT-1": {
  "scores": [4, 5, 3, 4, 4],
  "mean": 4.0,
  "threshold": 3.0,
  "passed": true,
  "timestamp": "2026-05-14T01:03:07"
 },
 "AT-3": {
  "answers": {
   "Q1": "right",
   "Q2": "sometimes they are like my siege team mates where its one on one and lose",
   "Q3": "their not very smart"
  },
  "passed": true,
  "timestamp": "2026-05-14T01:05:20"
 }
}
```


Test output 6: Training console output 1

```
-----------------------------------------
| time/              |          |
|    fps             | 97       |
|    iterations      | 2        |
|    time_elapsed    | 26       |
|    total_timesteps | 55004160 |
| train/             |          |
|    approx_kl       | 0.013505804 |
|    clip_fraction   | 0.255    |
|    clip_range      | 0.2      |
```

entropy_loss	-9.48
explained_variance	0.98
learning_rate	5e-05
loss	53.3
n_updates	214855
policy_gradient_loss	0.0112
std	0.415
value_loss	170

Appendix: github

https://github.com/fpb105/dissertation.git